

排球賽事標註、AI 串接與教練監看系統

實作總規格 v3.2

2026-08-07

目錄

0. 主 Agent 必須先理解的事情	4
1. 固定產品行為	5
1.1 標註快捷鍵與介面控制	5
快捷鍵自訂邊界	5
按鍵模式優先順序	5
1.2 Rally Mask 與狀態呈現	5
1.3 教練端不是單頁	6
1.4 AI 邊界	6
2. 實作與使用角度稽核：已修正問題	6
2.1 歷史稽核：v3.0 首次補齊的缺口	7
2.1.1 歷史稽核：v3.0 的實作修正	8
2.1.2 v3.2 實作／使用稽核新增修正	9
2.2 Schema 欄位來源標記	9
3. 三個團隊與 Schema Ownership	9
3.1 AI 團隊	9
負責	9
不負責	9
擁有 Schema	10
3.2 後端系統團隊	10
擁有	10
必須視為 Passthrough 的欄位	10
3.3 前端設計團隊	10
負責	10
不負責	10
3.4 Producer／Consumer Matrix	11
4. 最終 Repository 與技術架構	11
4.1 Monorepo	11
4.2 Web	12
4.3 API：Fastify + Yoga + Pothos + Prisma	12
Code-first 單一真實來源	13
4.4 Worker	13
4.5 Media	13
4.6 固定 Framework／Library 選型	13
4.7 iPad PWA 固定行為	14
4.7.1 本地 Docker 與 iPad 可信任 HTTPS	14
4.8 Serialization 分工	14
5. 容器與 Infrastructure	14
5.1 Baseline Containers	14
5.2 MiniIO Buckets	15
5.3 Durable Job	15

6. API 邊界：GraphQL、REST、WebSocket、FlatBuffers	15
6.1 GraphQL 適用	15
6.2 REST 適用	15
6.3 WebSocket 適用	16
6.4 FlatBuffers 適用	16
7. 統一時間與座標定義	16
7.1 統一時間欄位	16
7.2 Frame Position	17
7.3 Court Position	17
8. Full-session Live DVR 與 Lazy Playback	17
8.1 使用者體驗	17
8.2 Server Timeline	17
8.3 Playback Window 與完整 DVR	17
8.4 Client Buffer 策略	18
8.5 Playback Cursor 與 key point 對齊	18
8.6 Frame Step	18
9. Annotation 狀態機與完整 User Flow	19
9.1 狀態	19
9.2 完整操作	19
Z - 發球	19
Space - 擊球	19
<、>、? - 關閉 Rally 並記錄 Outcome	19
Enter - 提交	19
9.3 編輯與逐幀	20
9.4 UI 必備區域	20
10. Annotation Realtime Schema v2.0	20
10.0 協議版本	20
10.1 Client Command Envelope	20
10.2 Command Kinds	20
10.3 Ack	21
10.4 Conflict	21
11. GraphQL Schema 使用規格	21
11.0 Code-first 產生規則	21
11.1 BigInt	21
11.2 Query	22
11.3 Mutation	22
11.4 Rally Snapshot 範例	22
11.5 Feature Availability	22
12. PostgreSQL / Prisma 資料模型	23
12.1 核心分層	23
12.2 必要欄位與約束	23
12.3 JSONB 使用限制	23
13. Clip 與時間碼前處理合約	24
13.1 Submission → Clip Job	24
13.2 Canonical Clip	24
13.3 Timing Manifest	24
13.4 Clip Result	24
14. 外部 AI 串接：保留方法定義，不寫死模型細節	25
14.1 必要方法骨架	25
14.2 不在本規格寫死的事項	25
14.3 球歸屬與多球員重疊	25
14.4 Action 與群體 phase 待確認	25
15. AI Provider API 與 Python SDK	25

15.1 Provider Capabilities	25
15.2 AI Job Request	26
AI 欄位 ownership	27
15.3 Provider Accepted	27
15.4 Python SDK	27
16. AI Analysis Result Schema	27
16.1 結果範例	27
16.2 Passthrough 與 1:1 invariant	28
16.3 Actor observation frame	28
16.4 Association state	28
16.5 座標	29
16.6 Path Segment	29
17. AI Callback	29
17.1 Token	29
17.2 Progress / Failed	29
17.3 Completed Multipart	29
17.4 HTTP Semantics	30
18. FlatBuffers Overlay v1	30
18.1 原則	30
18.2 Chunk 內容	30
18.3 Web Manifest	30
19. 教練／裁判端 UX	31
19.1 Routes	31
19.2 現場頁	31
19.3 球路頁	31
19.4 回放頁	31
19.5 歷史與保存紀錄	31
19.6 Offline / Reconnect	32
20. 分析與統計	32
20.1 Baseline 一定可做	32
20.2 Identity 後可做	32
20.3 Action taxonomy 後才可做	32
20.4 API 回應	32
21. 後端／前端實作優先順序	33
Phase 0 — Contract Freeze 與 Scaffold	33
Contracts / SDK Agent	33
Backend Agent	33
Luna Frontend Agent	33
Exit	33
Phase 1 — Core Domain / Auth / DB	33
Phase 2 — Media Ingest / 完整 Timeline	33
Phase 3 — Annotation Vertical Slice	33
Phase 4 — Clip / Fake AI / SDK	34
Phase 5 — Coach Replay / Overlay	34
Phase 6 — Identity / Analytics	34
Phase 7 — Hardening	34
22. 三個 Subagent 的協作規則	34
22.1 固定 Ownership	34
22.2 第一輪任務	34
Agent A	34
Agent B	35
Agent C - Luna Frontend	35
22.3 每次交付格式	35
22.4 Main Agent Merge Gate	35

23. 主 Agent 可直接使用的 Prompt	35
24. Luna Worker 定義與建立 Prompt	36
24.1 建議 TOML	36
24.2 建立 Prompt	37
24.3 Project Config	37
25. 測試與 Definition of Done	37
25.1 Contract	37
25.2 Media / Time	38
25.3 Annotation	38
25.4 Clip / AI	38
25.5 Coach	38
25.6 第一版完成條件	38
26. 舊 MVP 可參考與禁止範圍	39
27. 尚待人類 / AI 團隊確認，但不阻塞 Baseline	39
28. 官方工程參考與選型依據	39
v3.2 交付註記	39
29. 最終欄位稽核與 Team Contract Catalog	40
29.1 三個團隊的 Schema 交付	40
29.2 前端送後端的 PlaybackCursor	40
29.3 後端解析後的 KeyPoint Anchor	40
29.4 Immutable Submission Snapshot	41
29.5 AI Job / Result 最小原則	41
29.6 已知待確認但不阻塞介面	41
30. v3.2 最終交付與啟動	41
Repository : volleyball-monitoring-ai	
主要讀者：負責統籌實作的主 PM / 架構 Agent，以及最多三個受委派 subagent。	
文件目的：把產品行為、跨團隊 Schema、媒體時間軸、外部 AI 串接、Nuxt UI、後端、容器與開發順序定成可實作的共同基線。	
核心邊界： 本 repository 不實作任何 AI 模型；只實作中央系統、影音流程、前端、外部 AI 介面、Fake Provider 與可由 GitHub 安裝的 Python SDK。	

0. 主 Agent 必須先理解的事情

這不是單純的影片播放器、標註工具或 AI Dashboard，而是一條完整且必須可追溯的資料鏈：

直播訊號

- 伺服器完整錄製與建立 **canonical timeline**
- 多人標註端在 **live** / 歷史回放畫面上建立人工 **key point**
- 人工確認並提交 **immutable rally submission**
- 後端依 **authoritative source time** 裁切影片並換算 **clip-local time**
- 外部 **AI** 透過固定 **Job Schema** 收件
- 外部 **AI** 透過 **callback** 回傳 **Analysis JSON + FlatBuffers overlay**
- 中央系統驗證、保存、正規化與聚合
- 教練／裁判端查看歷史、影片 **overlay**、2D 球路、熱點與統計

主 Agent 的第一責任是確保這條鏈中的 ID、時間、版本與資料語意不會斷裂。任何只完成 UI、只完成 API、只完成 DB table 或只完成 demo 的工作，都不能宣稱該流程已完成。

主 Agent 最多同時委派三個 subagent：

1. **Contracts / Python SDK Agent**
2. **Backend / Media / Infra Agent**
3. **Nuxt Luna Frontend Agent**

主 Agent 保留以下不可委派的最終責任：

- 產品語意與 Schema 變更核准。
- 跨目錄修改與 merge 衝突。

- 時間軸與 passthrough invariant 審查。
- 端到端驗收。
- 是否接受 subagent 提出的架構調整。

1. 固定產品行為

1.1 標註快捷鍵與介面控制

下列 command 語意與畫面按鈕是產品合約，不得由實作者自行改成其他語意。表中鍵位是預設值，使用者可以在設定選單修改實體快捷鍵；觸控按鈕與目前鍵盤綁定必須呼叫同一個 command path。

預設介面顯示	預設快捷鍵	固定語意	重要限制
Z 發球	Z	建立新 rally，並在目前已呈現影片 frame 建立第一個人工 service key point	已有未提交 rally 時不可另開新 rally
Space 擊球	Space	在目前已呈現影片 frame 建立一般 contact key point	播放器正在 seek、cursor stale 或位於 gap 時不可建立
< 左側得分	<	以單一 CLOSE_RALLY atomically 把目前 server-confirmed 最後 key point 標為 terminal，並將 rally-level outcome 設為 resolved/left	Command 必須帶 target_key_point_id ；若它已不是最後一點，回傳 revision conflict；不建立新時間或得分事件
> 右側得分	>	同樣 atomically 關閉 rally 並將 rally-level outcome 設為 resolved/right	同上；方向鍵仍只供逐幀／播放器控制
? 未知	?	同樣 atomically 關閉 rally 並將 rally-level outcome 設為 unknown/null	不是 pending ；可以提交 AI 但不納入勝負統計；不建立新時間或得分事件
Enter 提交	Enter	將目前 draft 建立成 immutable RallySubmission ，啟動裁切與 AI 串接	pending 不可提交； resolved 或 unknown 可提交

快捷鍵自訂邊界

- 使用者可以修改 annotation 與播放器 command 的實體鍵位；修改鍵位不得改變 command 語意、WebSocket command kind 或 server-side validation。
- 每個可用 command 必須保留一個有效鍵位。設定選單使用集中式 command registry、快捷鍵錄製器與衝突檢查，不得由各 component 各自監聽或保存。
- 設定選單必須提供「還原所有預設快捷鍵」。還原後回到本節表格與方向鍵的預設值。
- 相同 scope 不得有重複鍵位；瀏覽器保留鍵、文字輸入焦點、Dialog/Select scope 與平台鍵盤差異必須被辨識並清楚提示。
- Control deck、設定選單與快捷鍵說明必須以 TanStack Hotkeys **formatForDisplay** 顯示目前綁定，讓 macOS 使用符號、Windows/Linux 使用平台慣用標籤；不得自行拼接鍵帽文字。六個觸控動作不可被移除，且不受鍵盤自訂影響。

service 與 **contact** 只是人工 marker kind，不是 AI action label，也沒有人工 confidence。第一個 key point 被標成 **service** 只代表標註流程由 Z 開始；AI 是否輸出任何 action、action label 長什麼樣、是否有 confidence，均屬待 AI 團隊提供真實輸出後確認的 optional extension。

按鍵模式優先順序

1. 文字輸入、Dialog 或 Select 開啟時，禁止攔截全域快捷鍵。
2. **OPEN** 且存在 server-confirmed 最後 key point 時，左側得分、右側得分、未知等 **CLOSE_RALLY** command 依目前綁定觸發；預設為 <、>、?。
3. 前後 canonical frame／播放器移動 command 預設綁定 ←／→；使用者可改鍵，但該 command 不得變成得分或 annotation data mutation。
4. 其他狀態下播放器 command 只控制播放器，不可暗中改資料。
5. 每次 destructive action 都必須等待 server ack；optimistic UI 可顯示 pending，但不得把暫態當 canonical revision。

1.2 Rally Mask 與狀態呈現

- **OPEN**、**READY**：灰色半透明 mask，表示尚未提交且仍可修正。
- **SUBMITTED**：綠色半透明 mask，表示某個 immutable submission 已建立。
- 綠色不代表 AI 完成；裁切、排隊、處理中、完成、失敗以 mask 上的 status badge 顯示。
- Mask 上緣必須有文字狀態，不可只靠顏色：
 - 左側得分

- 右側得分
- ? 得分未知
- 回合進行中
- Mask 範圍永遠是 service key point 到 terminal key point；pre-roll/post-roll 只屬裁切範圍，不改變 mask。
- Key point marker 至少可辨識 **Z/service**、一般 **contact**、terminal、selected、possible duplicate、server pending。
- 顏色、線寬與動畫屬前端衍生視覺，不保存為 canonical DB 欄位。

1.3 教練端不是單頁

教練／裁判端採多頁資訊架構，iPad 使用底部導覽列；頂部狀態列可返回主選單並開啟過去保存的紀錄。

建議底部導覽：

1. 現場
2. 球路
3. 球員
4. 統計
5. 紀錄

頂部列至少包含：

- 返回賽事／場次選單。
- 場次、局數、比分與 live 狀態。
- 當前是否位於 live edge。
- 同步／離線狀態。
- Overlay 快捷設定。
- 進入歷史 rally、保存分析視圖與設定。

1.4 AI 邊界

外部 AI 團隊已有可利用的球場、球員、球、個體動作，以及可能存在的群體動作／比賽 phase work。本 repository：

- **不實作、不重訓、不替換這些模型。**
- 不規定 detector、tracker、action model 的架構。
- 不規定時間窗、IOU threshold、融合權重、補點參數或模型內部順序。
- 只規定目標導向的方法骨架、輸入、輸出、錯誤、版本與串接方式。
- action taxonomy、action confidence、群體 phase 的 label 與實際用途，需 AI 團隊提供真實輸出樣本後另行確認；baseline 不能硬編碼。

2. 實作與使用角度稽核：已修正問題

以下為原規格若直接實作會產生的問題，以及本文件採用的修正版。

嚴重度	問題	使用／實作風險	修正後基線
Critical	left/right 被當成永久隊伍 ID	換邊後歷史得分、熱點與球員統計錯隊	court_side 與 team_id 分離；每 rally 綁定當下的 side assignment
Critical	key point 直接信任 browser currentTime 或 currentTime * fps	HLS、VFR、seek、延遲與轉碼後會指向錯幀	Client 送 PlaybackCursor ；後端依 playback mapping 解出 authoritative PTS/time/frame
Critical	只提供幾分鐘回放 buffer	比賽後段無法返回開場；重新整理後資料消失	伺服器完整錄整場；前端只 windowed lazy-load，目前 buffer 幾分鐘但完整 timeline 可 seek
Critical	court_pos 被限制 [0,1]	發球員、救球、出界點會被 clamp 到場內	球場內以 0-1 為基準，但允許負值與大於 1；前端可裁切顯示，資料不可 clamp
Critical	multiple 同時表示共同參與與無法分辨	兩名重疊球員會被錯算成共同觸球	resolved_single 、 resolved_multiple 、 ambiguous 、 unresolved 分開；actors/candidates 分開
Critical	tracker ID 被當成跨 rally 球員 ID	同一整場球員統計會錯綁	track_id 只在單 rally／analysis run 內有效；中央另建 identity assignment
Critical	每次編輯都複製完整 revision snapshot	大量 DB 寫入與多人同步成本不必要	Draft 使用 mutable rows + operation log + revision；只有 Enter 建 immutable submission snapshot
High	? 與「尚未選得分」都用 null	UI 無法區分使用者已明確標未知	新增 score_resolution_state: pending/resolved/unknown ；? 後可提交，pending 不可提交

嚴重度	問題	使用／實作風險	修正後基線
High	舊流程以獨立 end-rally 動作建立另一個 timestamp	與使用者定義衝突，也會多一個虛假事件	CLOSE_RALLY 只標記指定的既有最後 key point 為 terminal 並保存 rally-level outcome；command 不含新時間
High	沒有 reopen／改 terminal 的流程	關閉 rally 誤按後只能刪整個 rally	Draft 提供 REOPEN_RALLY ；重開後以新的 CLOSE_RALLY 重新指定目前最後 key point 與 outcome；submitted 必須建立 correction draft
High	Rally 只有單一 status	annotation、clip、AI 狀態互相覆寫	annotation_status 與 processing_status 分離
High	submitted draft 可被原地修改	AI 結果無法知道對應哪個版本	submission 永久 immutable；修正建立新 submission，舊 job/result 標 superseded
High	Score 修正必定重跑 AI	只改勝方卻浪費推論	比對 submission content hash；只改 outcome 可重用 clip/AI geometry，重算分析聚合
High	AI Job 混入 court_definition 、模型 requirements、上傳 URI	AI request 像是在遠端配置 pipeline，耦合中央儲存	固定座標規格只寫一次；Job 僅含 clip、key points、outcome、callback 與 passthrough IDs
High	AI Result 的 x_m/y_m/x_norm/y_norm 多套命名	前後端容易混用或方向錯誤	對外只用 frame_pos 、 frame_bbox 、 court_pos ；公尺值由固定公式需要時衍生
High	AI actor 只有單一 actor_track_id	不能表達雙人攔網、共同事件或不確定候選	actors[] 與 candidates[] ；每個 contact event 具有 association state
High	Action label 被寫死	AI 團隊尚未確認 taxonomy，會被假規格綁住	action 是 optional extension，label 是 provider 提供字串，帶 taxonomy ID/version
High	Confidence 被視為必填	實際模型未必輸出可校準 confidence	所有 confidence optional；缺少時不得前端自行補 0 或 1
High	AI input/output IDs 沒有 passthrough 規則	callback 對不到 submission/key point	ai_job_id/rally_id/submission_id/annotation_revision/key_point_id 必須原樣回傳
High	一次 WebSocket 傳整段逐幀 JSON	iPad 記憶體、解析與 GC 壓力過大	JSON 管 metadata；FlatBuffers 管 per-frame overlay；HTTP lazy chunk，WS 只通知 ready/version
High	Clip URL 與 callback token 共用同一 bearer	SDK 若把 callback token 帶到物件儲存，會擴大 secret 暴露面	clip.download_url 是獨立、短期簽名 URL； callback.token 只可送到中央 callback，兩者不可混用且都不進 browser
High	callback token 真正一次性	網路重試會失敗	job-scoped、TTL 內可重試；每次 callback 有唯一 callback_id 做 idempotency
High	沒有 media discontinuity/gap	來源重連後時間軸假裝連續，key point 會錯段	capture timeline 保存 discontinuity；UI 顯示 gap，不允許 seek/mark 到缺檔區
High	GraphQL Int 保存 PTS／microseconds／size	GraphQL Int 僅 32-bit；JS Number 也有精度風險	使用 BigInt scalar，wire 一律 decimal string；DB 使用 PostgreSQL BIGINT
High	MinIO credential 交給 browser	物件儲存暴露	所有 S3 操作 server-side；前端只取得受權限控制的短期 URL 或同源 proxy route
Medium	Client 自行選 pre/post-roll	每個人裁切結果不一致	match/system clip profile 決定；submit 只提交 revision
Medium	只保存色彩，不保存狀態	改版後資料無法解讀	保存狀態 enum，顏色純 UI
Medium	Action 不存在時仍顯示 attack 指標	顯示空值或誤導數字	API 回傳 feature_availability 與 available_dimensions/metrics
Medium	統計把 rally win 當成某擊球直接得分	中間 attack 會被誤算 kill	分離 rally_outcome 與 optional direct_outcome ；baseline 只保證前者
Medium	WebSocket 斷線後只接續增量	錯過 revision 後狀態永久錯誤	reconnect 先比較 revision；有 gap 立即 GraphQL refetch snapshot，再訂閱增量
Medium	高頻操作透過 GraphQL subscription 與大型 snapshot	resolver/loading/serialization 不必要	GraphQL 管結構化查詢；自訂 WS 管 annotation command/revision/presence

2.1 歷史稽核：v3.0 首次補齊的缺口

嚴重度	缺口	修正
Critical	原始 PTS 在來源重連後可能從零或跳變，只有 discontinuity number 不足以關聯 key point	新增 CaptureEpoch ；所有 resolved marker 保存 capture_epoch_id + source_pts + capture_time_us 。capture_time_us 是整個 capture session 單調座標，raw PTS 只在 epoch 內有效。
Critical	terminal key point 可能是球落地、出界或無球員歸屬，卻被 schema 強迫一定有 actor	AI event 新增 no_player/unresolved 狀態， actors[] 可以為空； court_pos 也可為 null 並帶 quality flag。
Critical	AI result 的 court_side 容易被當成永久 team identity	AI 只輸出 court_side: left/right/unknown ；中央依 submission 的 side-assignment snapshot 解析 team_id 。

嚴重度	缺口	修正
Critical	clip key point 若引用 mutable draft ID，correction 後 callback 無法追溯	AI Job 只 引用 immutable RallySubmissionKeyPoint.id ；另保留 source_draft_key_point_id 供 audit。
High	得分欄位只有「誰得分」，沒有可重放的比分 ledger	新增 immutable PointAward ，保存 submission、scoring team、set score before/after 與 score_revision_before/after ；每 set 的 after revision 唯一，比分由 ledger 衍生。
High	同一場次可能停止再啟動錄影；單一 capture session 假設會使 timeline 斷裂	Match 可有多個 CaptureSession ，但 baseline 標註 room 指定一個 active capture；跨 capture replay 由 match timeline 聚合。
High	只保存 callback token，未定義 retry/idempotency	token 是 job-scoped 且有 TTL，可重試；每次 callback 傳 callback_id ，中央以唯一索引與 payload checksum 去重。
High	只定義 AI complete callback，無明確 accepted/failed 行為	SDK 與 REST 合約定義 accepted 、optional progress 、 failed 、 completed ；progress 不影響 correctness，failed 保存可重試分類。
High	AI event count 沒有明確與 key point 對應規則	Required invariant：每個 submission key point 恰好一個分析 event；segment 數通常為 max(event_count-1, 0) ，只有 service 且 terminal 的 service error 合法為 0 段。
High	frame_pos/court_pos 缺失時沒有語意	兩者為 nullable； quality_flags 與 association_state 表達缺失，禁止以 (0,0) 代表 unknown。
High	Overlay binary schema 若把 court 座標量化到 uint16，無法表示場外負值/大於 1	frame_pos/frame_bbox 可 uint16 量化； court_pos 使用 float32，允許場外值。
High	沒有 API authorization boundary	新增角色/permission 基線：administrator、operator、annotator、coach/viewer、integration service；MinIO 與 AI token 永不下發瀏覽器。
High	GraphQL source 與 SDL 可能雙向手改而漂移	Pothos source 唯一；SDL 為產物，CI 重生後必須 clean，並執行 breaking-change 檢查。
High	GraphQL scalar BigInt 若被 codegen 成 number 仍會精度損失	Web codegen 將 BigInt 映射為 string ；DB 為 BIGINT；JSON Schema 也用 decimal string。
Medium	使用者按鍵可能在播放器 seek 尚未完成時建立 marker	播放器 state machine 新增 cursor_status=ready/seeking/stale ；只有 ready cursor 可標記，其他狀態按鍵顯示阻擋原因。
Medium	多位標註者快速按同一碰撞會產生重複點，若自動去重可能誤刪真實短時間觸球	只標示 possible_duplicate ，不靜默合併；由人工刪除/合併。
Medium	保存 overlay preset 而沒有 schema/version	Saved view 保存 filter_schema_version 與 overlay_preset_version ；未知欄位忽略但保留原始 JSON。
Medium	歷史回放只有 media window、沒有完整 timeline cursor	URL/route 保存 capture_session_id + capture_time_us ，切換 window 後仍回到相同 canonical 時間。

2.1.1 歷史稽核：v3.0 的實作修正

嚴重度	缺口	修正後基線
Critical	GraphQL 文件使用 BigInt ，code-first builder 卻註冊 Int64	全部統一為 BigInt ；Web codegen 映射為 string 。
Critical	Annotation WebSocket 只定義自由格式 payload	每一個 Z / Space / CLOSE_RALLY / move / delete / reopen / Enter command 都具有 strict payload schema；close 必須帶 target ID 與 strict outcome。
Critical	DvrSegment.presentationStartUs 容易被誤認為 player/HLS presentation time	DB 改名 captureStartUs/captureEndUs ，只表示整場 canonical timeline。
High	SDK 內附的 FlatBuffers schema 與 canonical schema 欄位/版本不一致	SDK wheel 強制內含與 packages/contracts/flatbuffers/overlay.fbs 完全相同的檔案，CI byte-compare。
High	callback 文件使用 kind ，schema/SDK 使用 status	Callback metadata 統一使用 kind=processing/failed/completed 。
High	Score ledger 沒有 revision 序列	MatchSet.scoreRevision 與 PointAward.scoreRevisionBefore/After 形成 CAS ledger；after revision 在 set 內唯一。
High	AI ContactEvent 只保存 key point UUID 字串，沒有 DB relation	ContactEvent 外 鍵 指 向 immutable RallySubmissionKeyPoint ；仍由 domain validator 確同一 submission。
High	PlaybackWindow 與 frame-step 只存在文字範例，沒有 versioned schema	新增 request/response 分離的 media schemas，避免同一 oneOf 讓錯誤方向的 payload 也通過驗證。
High	Safari 原生 HLS 未必提供 segment ID	Browser cursor 不要求 segment ID；後端依 window mapping 與 sample index 解析。
Medium	Coach 底部導覽漏掉統計頁	固定五項：現場、球路、球員、統計、紀錄；頂部可返回場次選單與歷史。

2.1.2 v3.2 實作／使用稽核新增修正

嚴重度	缺口	修正後基線
Critical	Playback request 與 response 放在同一個 oneOf schema	拆成 playback-window-request / descriptor 、 playback-cursor / resolved-media-anchor 、 frame-step-request / canonical-frame-anchor ；OpenAPI 每個方向只引用正確 shape。
Critical	Annotation command 由 client 傳 user_id 並被當成可信身份	使用 HTTP/WebSocket upgrade 完成身份驗證並綁定 device session；command 不再攜帶可偽造的 user_id 。
Critical	CREATE_SERVICE 要求 server 既有 rally ID，但 Z 本身又負責建立 rally	前端在 Z 按下時產生 client UUID 作 rally_id 與 idempotency key；server 在同一 transaction 接受／建立 draft，不另做會改變時間點的前置 API。
High	ACK 只有自由格式 snapshot_patch ，前端拿不到 created key point / submission ID	command_ack.effects 明確回傳 created / terminal / deleted key point、submission、annotation status 與 score state；另廣播 typed rally snapshot。
High	PlaybackWindow 只回目前 window 範圍	Descriptor 新增整場 timeline_capture_start_us / end_us 及 has_more_before/after ，全場進度條不需把整場 media 塞進 client。
High	AI Provider capabilities 與 202 Accepted response 沒有正式 schema	新增 capabilities.schema.json 與 job-accepted.schema.json ，Python SDK 也提供 typed models。
High	AI result 只檢查數量，未保證 segment 連接相鄰 event 或 A/B 一致	SDK validator 要求 event/segment index 連續、segment 引用相鄰 key point、A/B 等於對應 contact event representative positions、terminal flag 一致。
High	observed ball 可以沒有座標，而 missing 卻可殘留座標	observed/interpolated 必須帶 sample frame 與 frame_pos ；missing 兩者皆不得帶。
High	Submission 可保存 PENDING ，且 clip policy 只有版本沒有實際 roll 值	Submission 使用只含 RESOLVED / UNKNOWN 的 enum，並 snapshot clip_pre_roll_us / clip_post_roll_us 及 service/terminal IDs。
High	MediaAsset 在 UPLOADING 狀態就強制要求 checksum/size	byte_length 與 sha256 在 upload 完成前可 null；轉 READY 時 domain transaction 必須補齊。
Medium	Browser OverlayChunk 內嵌完整 OverlaySequence	Chunk schema 改成獨立 SoA payload，只保留 chunk 範圍與陣列，不重複整段 metadata。

2.2 Schema 欄位來源標記

所有跨系統欄位表必須標示以下 ownership，而不是只列型別：

- **USER_INPUT**：由標註者或管理者明確輸入。
- **CLIENT_OBSERVED**：瀏覽器觀測值，例如 PlaybackCursor；不可當 authoritative。
- **BACKEND_DERIVED**：後端從 mapping/assignment/ledger 推導。
- **PREPROCESS_DERIVED**：裁切與 sample mapping 產生。
- **AI_GENERATED**：外部 AI 分析輸出。
- **PASSTHROUGH**：上游建立，接收端必須原樣回傳/關聯。
- **CONSTANT**：固定於版本化規格，不在每個 request 重複傳送。
- **OPTIONAL_EXTENSION**：只有 capability 聲明支援時才可用。

主 Agent 在新增任何 public field 前，必須先寫出 producer、consumer、source marker、nullability、versioning 與失敗語意。

3. 三個團隊與 Schema Ownership

3.1 AI 團隊

負責

- 以 SDK / HTTP 接收 AI Job。
- 使用既有 AI work 產生 tracks、contact events、participants、A/B segments 與 overlay。
- 保持所有 PASSTHROUGH 欄位不變。
- 呼叫中央 callback。

不負責

- 直播採集、整場錄影、時間軸或影片裁切。
- Nuxt UI、GraphQL、PostgreSQL 或 MinIO 的中央資料模型。

- 跨 rally 球員身份。
- 中央統計公式與教練頁面。

擁有 Schema

AI 團隊不單方面擁有任何 public schema。它與後端共同消費：

- `packages/contracts/ai/job.schema.json`
- `packages/contracts/ai/result.schema.json`
- `packages/contracts/ai/callback.schema.json`
- `packages/contracts/flatbuffers/overlay.fbs`

Schema 由主 Agent 核准；AI 團隊透過 Python SDK 使用。

3.2 後端系統團隊

包含中央 API、影音採集、完整 DVR、裁切、AI orchestration、callback ingest、MinIO、PostgreSQL、Redis 與 worker。

擁有

- Prisma schema/migrations。
- Pothos code-first schema modules、resolver 與產生後的 SDL。
- REST/OpenAPI。
- WebSocket command/event protocol server 實作。
- 媒體 timeline 與 PlaybackCursor resolution。
- Clip preprocessing 與 AI Job 建立。
- Result 驗證、正規化、分析聚合與 artifact 管理。
- 容器與部署。

必須視為 Passthrough 的欄位

AI request 建立後，下列值在 AI result/callback 不得改寫：

- `ai_job_id`
- `rally_id`
- `rally_submission_id`
- `annotation_revision`
- `clip_id`
- 每個 `key_point_id`
- 每個 key point 的 `sequence_index`
- `marker_kind`
- `is_terminal`

3.3 前端設計團隊

負責

- Nuxt 多頁式標註端、教練端與歷史紀錄。
- iPad landscape 互動。
- Full-session timeline + lazy playback windows。
- 快捷鍵 state machine 與灰／綠 mask。
- GraphQL query/mutation client。
- Annotation WebSocket command/reconciliation。
- Video + Canvas overlay 與 2D court。
- Feature availability／資料品質／樣本數的正確呈現。

不負責

- 自行換算 authoritative source PTS/frame。
- 自行從球影像座標推導 court A/B。
- 自行認定 action taxonomy。
- 直接存取 MinIO credential。
- 把 track ID 當 player ID。

3.4 Producer / Consumer Matrix

Contract	Producer	Consumer	傳輸
Match / Set / Rally structured data	Backend	Web	GraphQL
Annotation command	Web	Backend	WebSocket ; GraphQL fallback
Annotation ack/snapshot/conflict	Backend	Web	WebSocket
Timeline availability/live edge	Backend	Web	GraphQL + WebSocket delta
Playback window	Backend media	Web player	REST + HLS/fMP4/Range
Clip job	Backend	Worker	PostgreSQL durable job
AI Job	Backend AI gateway	External AI	REST JSON
AI callback metadata	External AI SDK	Backend	REST JSON/multipart
Full per-frame overlay	External AI	Backend	FlatBuffers multipart
Overlay manifest/chunks	Backend	Web	REST JSON + FlatBuffers
Analysis query	Backend	Web	GraphQL
Presence/soft lock	Backend	Web	WebSocket

4. 最終 Repository 與技術架構

4.1 Monorepo

```
volleyball-monitoring-ai/
├── AGENTS.md
├── README.md
├── package.json
├── bunfig.toml
├── .env.example
├── .codex/
│   ├── config.toml
│   └── agents/
│       ├── contracts-sdk-worker.toml
│       ├── backend-worker.toml
│       └── luna-worker.toml
├── docs/
│   ├── MASTER_IMPLEMENTATION_SPEC.md
│   ├── SYSTEM_SPEC_V3_2.pdf
│   ├── MAIN_AGENT_PROMPT.md
│   ├── LUNA_WORKER_SETUP_PROMPT.md
│   ├── requirements-matrix.md
│   ├── progress.md
│   ├── open-decisions.md
│   └── adr/
├── packages/
│   ├── contracts/
│   │   ├── annotation/
│   │   ├── media/
│   │   ├── ai/
│   │   ├── openapi/
│   │   ├── flatbuffers/
│   │   └── graphql/
```

```

├── fixtures/
├── db/
│   ├── prisma/schema.prisma
│   ├── prisma/migrations/
│   └── src/
├── sdk/
│   ├── pyproject.toml
│   ├── src/volleyball_monitoring_ai/
│   ├── examples/
│   └── tests/
├── web/
│   ├── app/
│   ├── graphql/
│   ├── tests/
│   └── nuxt.config.ts
├── server/
│   ├── src/graphql/
│   ├── src/domain/
│   ├── src/rest/
│   ├── src/realtime/
│   └── tests/
├── worker/
│   ├── src/roles/
│   └── tests/
├── examples/
│   └── fake_ai_provider/
├── infra/
│   ├── compose.yaml
│   ├── docker/
│   ├── mediamtx/
│   └── k8s/
├── scripts/
└── .github/workflows/ci.yml

```

採 top-level **server/** 與 **worker/**，避免同一文件同時出現 **backend/api**、**backend/worker** 與另一套 root 路徑。

4.2 Web

- Nuxt 4、Vue 3、TypeScript strict。
- UI 可以參考舊 MVP 使用的 Nuxt / Vant / Tailwind / Motion / Lucide 組合，但不得沿用其 domain flow、memory store 或手動事件模型。
- Vant 4 用於 iPad/mobile interaction；Tailwind 4 建立 design tokens；Motion for Vue 只用於必要狀態與球路動畫。
- GraphQL Code Generator **client-preset** 由產生後 SDL 與前端 operations 建立 **TypedDocumentNode**；禁止手寫另一套 domain interface。
- Video element 上疊透明 Canvas；影片本身不逐 frame 畫入 Canvas。

4.3 API : Fastify + Yoga + Pothos + Prisma

中央 API 採獨立 TypeScript service，不把長生命週期 WebSocket、callback 大檔與 FFmpeg 工作塞入 Nuxt/Nitro。

- **Fastify**：HTTP server、REST routes、WebSocket upgrade、lifecycle 與 health endpoints。
- **GraphQL Yoga**：GraphQL over HTTP 執行層。
- **Pothos**：唯一 GraphQL code-first schema source；**builder.toSchema()** 產生標準 **GraphQLSchema** 交給 Yoga。
- **Pothos Prisma plugin**：建立 Prisma-backed object/relations，並利用 selection information 避免常見 N+1。
- **Prisma 7 + PostgreSQL**：canonical persistence；使用 **prisma.config.ts**、**prisma-client** generator 與 PostgreSQL driver adapter。
- **GraphQL Code Generator**：從 Pothos schema 輸出 checked-in SDL，再由 client preset 產生 Nuxt typed operations。
- **JSON Schema/Zod**：REST、WebSocket 與 callback validation。
- **Redis**：presence、soft lock、pub/sub fan-out；不得放 durable canonical state。
- **pg-boss 或等價 PostgreSQL-backed queue**：durable media/AI jobs。

Code-first 單一真實來源

server/src/graphql/**/*.ts (Pothos source)
→ builder.toSchema()
→ packages/contracts/graphql/schema.graphql
→ GraphQL Inspector / CI contract diff
→ web/codegen.ts
→ web/app/gql/** typed documents

規則：

1. 不可手改 **graphql/schema.graphql**。
2. Prisma model 不等於 GraphQL public contract；只有 Pothos 明確 expose 的欄位會公開。
3. GraphQL resolver 必須呼叫 domain/service layer，不能繞過 annotation、time mapping 或 submission invariant。
4. 高頻 annotation collaboration 使用專用 WebSocket command protocol；GraphQL mutation 只作 fallback，不強迫使用 GraphQL subscription。
5. Binary、影片、FlatBuffers、AI multipart callback 不進 GraphQL。

4.4 Worker

同一個 worker image 可以依 command/environment 執行不同角色：

- media-indexer
- playback-packager
- clip-worker
- ai-dispatcher
- analysis-ingest

它們共享 **packages/db** 與 **packages/contracts**，但每個 worker 只能 claim 自己的 job type。FFmpeg subprocess 必須有 timeout、cancel、stderr capture、temporary directory quota 與 idempotent output key。

4.5 Media

- MediaMTX：接收來源、live LL-HLS/WebRTC、fMP4 recording 與基礎 playback。
- FFmpeg/ffprobe：索引、archive playback window、canonical clip、preview 與 sample mapping。
- MinIO：完整錄影片段、playback windows、clips、AI artifacts。
- PostgreSQL：capture epochs、segment index、available ranges、mapping version、job state。
- 標註用播放 profile 必須是可建立 deterministic media-time → capture-time mapping 的單一 rendition；若未來開放 ABR，每個 rendition 必須各自有 mapping，不能共用 **currentTime × FPS**。

4.6 固定 Framework / Library 選型

第一版 starter 固定以下基線，避免主 Agent 與三個 subagent 各自選一套：

層	選型	用途
Runtime / Package	Bun 1.3.14	workspace install、script、server/worker runtime、Docker image
Web UI	Nuxt 4、Vue 3、TypeScript Vant 4、Tailwind CSS、Motion-V、Lucide、VueUse	Annotation 與 Coach 多頁 PWA 觸控元件、版面、物理動畫、icon 與 composable
PWA	@vite-pwa/nuxt	standalone manifest、service worker、更新提示、iPad 加入主畫面
GraphQL Client Media Client API Server	URQL + GraphQL Code Generator native Safari HLS + HLS.js fallback Fastify 5 + GraphQL Yoga 5	讀取結構化 domain 與 generated type live/archive HLS、bounded client buffer 單一 HTTP server 內同時掛 GraphQL、REST 與 WebSocket upgrade
Code-first GraphQL Data	Pothos + Prisma plugin Prisma 7 + PostgreSQL 17	TypeScript code-first schema；SDL 只作 CI 產物 domain、revision、media index、submission、job 與 analysis metadata
Job / Realtime Object Storage	pg-boss + Redis MinIO S3	durable worker job 與 WS fan-out / presence raw recording、DVR、canonical clip、analysis 與 overlay artifacts
Media	MediaMTX + FFmpeg/ffprobe	ingest、LL-HLS、完整錄製、index、playback window 與裁切
Edge AI Integration	Traefik 3.7.9 Python 3.11+ SDK	本地 Docker 同源 routing、WebSocket 與 HLS 反向代理 外部 AI team 驗證 Job / Result、下載 clip、callback 與 FlatBuffers

不採 Caddy。瀏覽器使用 Traefik 同源路徑 `/graphql`、`/api/v1`、`/ws`、`/hls`，避免把 `localhost` 編入 iPad bundle。MinIO credentials、AI provider bearer 與 callback token 不可進前端。

4.7 iPad PWA 固定行為

- PWA **display=standalone**、landscape-first、支援 safe-area 與 Apple touch icon。
- iPad 安裝與 Service Worker 必須使用可信任的 HTTPS origin；LAN 上的純 HTTP IP 只能作一般瀏覽器預覽，不得當作 PWA 驗收環境。
- **orientation=landscape** 只是一個 manifest 偏好，UI 仍要在直向時顯示「請旋轉 iPad」提示，不得依賴瀏覽器一定鎖定方向。
- 頂部狀態列與底部五分頁導覽在 standalone 模式仍完整可用。
- Service worker 可 cache app shell 與已讀分析資料；GraphQL mutation、annotation WebSocket、live/DVR media、callback 與 signed artifact 一律 NetworkOnly。
- Offline 時可以查看已 cache 的歷史資料，但未獲 server ACK 的 annotation 只能顯示 local pending，不得冒充 canonical revision。
- PWA 更新使用 auto-update 提示；執行中的 annotation 不可無提示強制 reload。

4.7.1 本地 Docker 與 iPad 可信任 HTTPS

本地 Docker 仍由 Traefik 統一入口。Desktop 可先用 HTTP 檢查服務，但 iPad「加入主畫面」、Service Worker 與正式 PWA 驗收必須使用可信任 HTTPS：

1. 為 LAN hostname 或 IP 建立本地開發憑證；starter 提供 **scripts/generate-local-tls.sh**，預設使用 **mkcert**。
2. 將本地 CA 安裝到 iPad，並在 iOS 的憑證信任設定中啟用完整信任。
3. Traefik 的 **web** entrypoint 只做 HTTP 到 HTTPS redirect；PWA、GraphQL、REST、WebSocket 與 HLS 全部走 **websecure** 同源入口。
4. 憑證私鑰與本地 CA 不得提交 Git；repository 只保存產生腳本與範例設定。
5. 若部署到正式 DNS，改用組織既有 PKI 或 ACME，不沿用本地 CA。

4.8 Serialization 分工

類型	格式	原因
一般資料查詢／mutation	GraphQL JSON	關聯查詢、code-first contract、Nuxt type safety
Media／binary／callback	REST	range、stream、multipart 與大型 payload 不適合 GraphQL
即時 annotation command／status	WebSocket JSON	command ID、revision、低延遲、重連
AI event summary	JSON Schema	可讀、易驗證、資料量有限
逐幀 tracking/ball/action	FlatBuffers	避免巨大 JSON object、降低 iPad parse/GC 成本

5. 容器與 Infrastructure

5.1 Baseline Containers

Container	責任	是否有 durable state
web server	Nuxt UI/SSR/SPA GraphQL、REST、WS、auth、domain command	否 否
worker-media-indexer / worker-playback / worker-clip	index、playback window、clip、FFmpeg	local spool 只是暫存
worker-ai-dispatcher / worker-analysis-ingest	submit、retry、callback ingest／normalize	否
mediamtx postgres minio redis minio-init	ingest/live/record canonical metadata、revision、job、analysis media、clip、analysis artifact presence、soft lock、fan-out 建 buckets/lifecycle	recording spool 暫存 是 是 否，可重建 否

Baseline 不建立 AI 推論容器。開發環境可另啟動 **fake-ai-provider**，它只驗證 contract 與產生 fixtures。

5.2 MinIO Buckets

```
raw-media/  
  matches/{match_id}/captures/{capture_id}/segments/{discontinuity}/{segment_id}.mp4
```

```
playback/  
  matches/{match_id}/captures/{capture_id}/windows/{window_id}/...
```

```
rally-media/  
  matches/{match_id}/rallies/{rally_id}/submissions/{submission_id}/  
    canonical.mp4  
    preview.mp4  
    timing-manifest.json
```

```
analysis-artifacts/  
  matches/{match_id}/rallies/{rally_id}/runs/{analysis_run_id}/  
    result.json  
    overlay-sequence.fb  
    overlay-manifest.json  
    chunks/{index}.fb
```

物件 key 不作資料庫主鍵。DB 永遠保存 bucket、object key、checksum、byte size、MIME、schema version 與 lifecycle state。

5.3 Durable Job

- Clip、playback package、AI submit、artifact ingest 都必須是 DB-backed durable job。
- Worker claim 使用 queue library 的 PostgreSQL transaction/lease；不得只用 Redis list。
- 每個 job 必須有：
 - id
 - job_type
 - status
 - deduplication_key
 - attempt_count
 - max_attempts
 - available_at
 - leased_until
 - last_error_code/message
 - created_at/updated_at/completed_at
- Worker crash 後 lease 到期可由另一 worker 接手。

6. API 邊界：GraphQL、REST、WebSocket、FlatBuffers

6.1 GraphQL 適用

- Match、team、player、roster、set。
- Capture session metadata 與完整 timeline availability。
- Rally list/detail、submission history、job status。
- Analysis、filter、statistics、saved view。
- 管理設定與 track identity assignment。
- Annotation snapshot/refetch 與低頻 fallback mutation。

GraphQL 不傳：

- MP4、HLS segment、FlatBuffers、callback multipart。
- 每 frame tracking array。
- MinIO credential。

6.2 REST 適用

```
POST /v1/media/playback-windows  
GET /v1/media/playback-windows/{id}  
GET /v1/media/playback-windows/{id}/manifest.m3u8
```

```
GET /v1/media/playback-windows/{id}/media.mp4
POST /v1/media/resolve-cursor
POST /v1/media/frame-step

GET /v1/ai/jobs/{ai_job_id}/clip
POST /v1/ai/callback/{ai_job_id}
GET /v1/analysis-runs/{id}/overlay-manifest
GET /v1/analysis-runs/{id}/overlay-chunks/{index}

GET /health/live
GET /health/ready
```

6.3 WebSocket 適用

單一連線可加入 match room，訊息分為：

- `annotation.command`
- `annotation.ack`
- `annotation.conflict`
- `annotation.snapshot_changed`
- `presence.update`
- `soft_lock.update`
- `timeline.live_edge`
- `timeline.segment_added`
- `processing.status_changed`
- `analysis.ready`

WebSocket 不廣播完整逐幀 overlay，也不應每次小操作廣播整場大型 snapshot。

6.4 FlatBuffers 適用

只負責：

- 每 frame 的 track observation。
- frame bbox、foot frame position、court position。
- ball frame position。
- optional action dictionary index。
- flags / optional quantized confidence。

Event、rally、team、player、filter 仍使用 JSON/GraphQL。

7. 統一時間與座標定義

7.1 統一時間欄位

時間分成四層，欄位名稱不得互換：

1. **來源 epoch 時間**：`capture_epoch_id + source_pts + source_time_base`。來源重連或 PTS 重置時建立新 epoch；raw PTS 只在 epoch 內有意義。
2. **整場 canonical 時間**：`capture_time_us`，從 capture session 起點單調增加，資料庫使用 PostgreSQL `BIGINT`，wire 使用十進位字串。
3. **整場 canonical frame**：`capture_frame_index`，由 sample index 建立；瀏覽器不得以 `currentTime × FPS` 推算。
4. **裁切片段局部時間**：`clip_time_us`、`clip_pts`、`clip_frame_index`，均由前處理服務產生，AI 只使用這組局部時間。

Browser 只送 `PlaybackCursor` 觀測值。後端依 `playback_window_id + mapping_version + player_media_time_us` 查 window mapping 與 sample index，解析為 authoritative anchor。`requestVideoFrameCallback().mediaTime` 是首選，`currentTime` 只作 fallback；兩者都仍是 client observation，不直接寫進 key point canonical 欄位。

GraphQL `Int` 只有 32-bit，不能承載 microseconds、PTS、frame count 或 byte size。所有 64-bit wire 欄位使用自訂 `BigInt` scalar 並序列化為 decimal string；前端 codegen 映射為 `string`，不可映射為 JavaScript `number`。

7.2 Frame Position

```
{"x": 0.42, "y": 0.63}
```

- `frame_pos` 原點是影像左上。
- `x/y` 通常位於 `[0,1]`。
- `frame_bbox` 使用 `x1/y1/x2/y2` 正規化。
- 前端按實際 video content rect 換算，不可把 letterbox/pillarbox 也當影像區。

7.3 Court Position

```
{"x": 0.31, "y": 0.72}
```

固定唯一規格：

- `x=0`：左側端線；`x=1`：右側端線，對應 18 m 長度。
- `y=0`：canonical 俯視球場圖的上側邊線；`y=1`：下側邊線，對應 9 m 寬度。此方向不依賴攝影機畫面。
- 球場內 `[0,1] × [0,1]`。
- 發球區、救球區、出界點允許 `<0` 或 `>1`。
- 不因換邊翻轉 canonical data；前端可依教練視角 transform。
- 公尺值需要時：`x_m = x * 18`、`y_m = y * 9`。
- A/B 由 AI 已處理的 footprint proxy / terminal representative point 產生；中央與前端不再投影。

8. Full-session Live DVR 與 Lazy Playback

8.1 使用者體驗

- 標註端進入場次後預設位於 live edge，持續播放聲音與影像。
- 底部 timeline 範圍是整個 capture session，從第一段可用影像到 live edge。
- 使用者可像 YouTube Live 一樣拖到任何已保存時間。
- 遠端歷史影片不一次下載整場；播放器只保留目前 window 與前後相鄰 window。
- 使用者停留在歷史回放時，伺服器仍持續錄 live；WebSocket 持續更新 live edge。
- 按「回到現場」時切回 live rolling window。
- 若某區段缺檔，timeline 顯示 gap，不允許在 gap 建 marker。

8.2 Server Timeline

GraphQL `captureTimeline` 回傳 metadata，而不是 media bytes：

```
{
  "capture_session_id": "...",
  "timeline_version": 87,
  "capture_start_time_us": "0",
  "live_edge_capture_time_us": "7132456000",
  "available_ranges": [
    {"start_us": "0", "end_us": "2410000000", "discontinuity": 0},
    {"start_us": "2416500000", "end_us": "7132456000", "discontinuity": 1}
  ]
}
```

UI 可用這些 range 繪完整時間軸；不需要先下載所有 segment list。

8.3 Playback Window 與完整 DVR

Server 保存整場，client 只取得 bounded playback window。Live manifest 與 archive window 都映射到同一條 `capture_time_us`；播放器的 `currentTime` 原點可以不同，因此每個 descriptor 必須攜帶 mapping origin。

Request：

```
{
  "capture_session_id": "018f...",
  "mode": "archive",
  "target_capture_time_us": "2823123000",
  "requested_back_us": "90000000",
  "requested_forward_us": "120000000"
}
```

Response：

```
{
  "playback_window_id": "0190...",
  "capture_session_id": "018f...",
  "mode": "archive",
  "mapping_version": 3,
  "window_capture_start_us": "2733123000",
  "window_capture_end_us": "2943123000",
  "presentation_origin_capture_us": "2733123000",
  "target_player_media_time_us": "90000000",
  "manifest_url": "/api/v1/media/playback-windows/0190.../manifest.m3u8",
  "expires_at": "2026-08-07T05:00:00Z"
}
```

`capture_time_us = presentation_origin_capture_us + player_media_time_us` 只是一階 mapping；後端仍須以 sample index 吸附到實際 frame。HLS playlist 可使用 program date time 協助跨 rendition / 窗口對齊，但 canonical 資料仍以自己的 capture/sample index 為準。

預設 window 可設 3-5 分鐘，這只是 client loading 策略，不是回放限制。接近 window 邊界時 lazy prefetch 下一個 window；使用者在歷史回放時，live 錄製與 live-edge 更新持續進行。

8.4 Client Buffer 策略

- **live**：使用 LL-HLS rolling manifest，只保存最近幾分鐘的 segment references。
- **archive**：下載固定 window；接近 20-30 秒邊界時 prefetch 相鄰 window。
- 記憶體只保留 current/previous/next window metadata；已遠離的 MediaSource buffer 與 overlay chunk 必須釋放。
- 不在背景維持第二支 live video；以 WebSocket live edge 更新替代，避免雙倍頻寬。
- 回到 live 時再 load live manifest。

8.5 Playback Cursor 與 key point 對齊

前端在鍵盤或觸控按鈕被觸發時，送「最後一個真正呈現的影片 frame」觀測：

```
{
  "playback_window_id": "0190...",
  "mapping_version": 3,
  "player_media_time_us": "90167234",
  "observation_source": "request_video_frame_callback",
  "presented_frames": "1842",
  "seek_generation": 27,
  "cursor_status": "ready"
}
```

- `observation_source` 可為 `request_video_frame_callback` 或 `current_time_fallback`。
- Safari 原生 HLS 不保證可暴露 segment ID，因此 `segment_id` 不可設為 browser 必填欄位。
- `cursor_status=seeking/stale/gap` 時，Z/Space 必須被 UI 阻擋。
- `seek_generation` 防止 seek 完成前的舊 callback 被誤用。

後端解析後回：

```
{
  "capture_epoch_id": "0190...",
  "source_pts": "169387400",
  "source_time_base": {"num": 1, "den": 60000},
  "capture_time_us": "2823290234",
  "capture_frame_index": "169228",
  "resolved_player_media_time_us": "90166900",
  "mapping_version": 3,
  "snap_distance_us": "334",
  "timing_precision": "frame_exact"
}
```

這個 server resolve 結果才可寫入 KeyPoint。無法解析時必須拒絕 command，不得退化成 client clock。

8.6 Frame Step

編輯 key point 時，前端送 key point ID 與方向：

```
{"key_point_id":"...", "direction":"next", "step_count":1}
```

後端依 sample index 回傳前／後一個 frame 的 authoritative cursor。播放器 seek 到對應 media time；marker 更新仍走 revision command。

9. Annotation 狀態機與完整 User Flow

9.1 狀態

OPEN

- | < CLOSE_RALLY(target, resolved/left) → READY
- | > CLOSE_RALLY(target, resolved/right) → READY
- | ? CLOSE_RALLY(target, unknown/null) → READY

READY

- | reopen → OPEN (清除 terminal 與 outcome，之後重新 CLOSE_RALLY)
- | Enter → SUBMITTED

SUBMITTED

- | correction → 新 draft / 新 submission

VOIDED

處理狀態另存：IDLE / CLIP_QUEUED / CLIPPING / AI_QUEUED / AI_PROCESSING / INGESTING / COMPLETED / FAILED / SUPERSEDED。

9.2 完整操作

Z - 發球

1. 播放器 cursor 必須 **ready** 且不在 gap。
2. Client 送單一旦可重試的 **CREATE_SERVICE_KEY_POINT** command；**marker_kind=service** 由 command 語意固定。
3. Server 解析 authoritative frame，建立 RallyDraft 與 sequence 0 key point。
4. Timeline 開始顯示灰色 open mask。
5. 若已有 active draft，拒絕，不自動關閉前一 rally。

Space - 擊球

1. 僅 OPEN 可新增。
2. 每次送一個 **contact** marker。
3. Server 依 canonical time 排序並產生 revision。
4. 多人幾乎同時打同一球時保留兩點並標 **possible_duplicate**，不靜默刪除。

<、>、? - 關閉 Rally 並記錄 Outcome

1. UI 顯示三個按鈕：< 左側得分、> 右側得分、? 未知。硬體鍵盤使用實際 <、>、?；方向鍵保留給逐幀。
2. Client 從目前 server-confirmed snapshot 取得最後一個有效 key point ID，作為 **target_key_point_id**，並送單一 **CLOSE_RALLY** command。
3. < 傳 **resolved/left**、> 傳 **resolved/right**、? 傳 **unknown/null**。Outcome 只屬於 rally，不是 key point。
4. Command 不含 playback cursor、capture time、frame 或 score event；terminal 時間／frame 就是被選定 key point 既有的 authoritative anchor。
5. Server 在同一 transaction 驗證 target 於 **base_revision** 仍是最後一個未刪除 key point，將它標為 terminal，保存 outcome 並把狀態改為 **READY**。
6. 若另一協作者已新增點，回 **REVISION_CONFLICT**／**CLOSE_RALLY_TARGET_NOT_LAST**；前端 refetch 後讓使用者再決定。
7. **left/right** 指當下球場左右側，不是永久 team identity；Server 依 immutable submission 的 side-assignment snapshot 解析 team ID。
8. **pending** 與 **unknown** 不同：pending 只允許 open draft；unknown 代表使用者已明確選? 且可以提交。

Enter - 提交

Server 以單一 transaction：

1. 驗證至少一個 service、恰好一個 terminal、terminal 為最後有效 key point。
2. 驗證 score state 是 **resolved** 或 **unknown**。
3. 鎖定 draft revision 並建立 immutable **RallySubmission**。

4. 複製 immutable submission key points、side assignment、score resolution、clip policy 與 content hash。
5. **resolved** 才建立 **PointAward**；**unknown** 不改比分 ledger。
6. 建立 ClipJob 與 Outbox event。
7. 廣播 submission / processing 狀態；UI mask 轉綠。

提交後不可原地修改。修正必須建立 correction draft 並新建 submission，保留 supersedes 關係。

9.3 編輯與逐幀

- 灰色 mask 可新增、刪除、移動、重開 rally；要改 terminal 或 outcome 時先 **REOPEN_RALLY**，再以新的 **CLOSE_RALLY** atomically 關閉。
- 方向鍵在 key point edit mode 呼叫 server frame-step API；後端依 sample index 取得前後 sample，再更新 marker。
- 綠色 mask 唯讀；開啟修正時從 submission 建立新 draft。
- 拖曳 marker 時可用短暫 soft lock 提示其他協作者，但 canonical concurrency 仍以 revision/CAS 為準。

9.4 UI 必備區域

- 頂部：場次、來源健康、LIVE 狀態、與 live edge 距離、WebSocket、協作者、來源 timecode。
- 中央：有聲 video、回到 LIVE、播放/暫停、逐幀、倍速。
- 多軌 timeline：完整 capture range、gap、playhead、key points、rally masks、score strip、processing badges。
- 底部固定 control deck：發球、擊球、左側得分、右側得分、未知、提交六個觸控動作，顯示目前快捷鍵（預設 **Z**、**Space**、**<**、**>**、**?**、**Enter**）並依 state 啟停；不得顯示獨立結束控制。
- Inspector：選取 key point 時顯示 canonical time、frame、建立者、revision、duplicate/precision 資訊。

10. Annotation Realtime Schema v2.0

10.0 協議版本

Annotation Realtime Protocol 的正式版本為 **2.0.0**。這是 breaking change：移除 v1.1 的獨立 terminal 與後續 score 兩階段 command，改由一個 **CLOSE_RALLY** atomically terminalize 目前 server-confirmed 最後 key point 並保存 rally-level outcome。任何 v1.x annotation message 都不得被 2.0 consumer 靜默當成相同語意。

WebSocket 用於高頻 annotation command、ack、revision、presence 與處理通知。GraphQL 提供 snapshot/refetch，不承擔逐次按鍵。

10.1 Client Command Envelope

```
{
  "schema_version": "2.0.0",
  "command_id": "0190...",
  "room_id": "match:...:capture:...",
  "rally_id": "0190...",
  "base_revision": "27",
  "kind": "CLOSE_RALLY",
  "payload": {
    "target_key_point_id": "0190...",
    "score_resolution": "resolved",
    "scoring_court_side": "left"
  }
}
```

user_id 與 **device_session_id** 不由每一個 command 自報。HTTP/WebSocket upgrade 先驗證登入身份並綁定 device session；server 寫入 operation log 時使用 connection context。**CREATE_SERVICE_KEY_POINT** 的 **rally_id** 由 client 在按 Z 當下產生 UUID，讓「建立 rally + 解析該 frame + 建立 service marker」保持單一 idempotent command，不需要先呼叫另一個會造成時間漂移的 API。

所有 revision、time、PTS、global frame 在 JSON wire 使用 decimal string。

10.2 Command Kinds

kind	主要 payload	說明
CREATE_SERVICE_KEY_POINT	playback_cursor	Z；建立 rally 與 service marker
CREATE_CONTACT_KEY_POINT	playback_cursor	Space
CLOSE_RALLY	target_key_point_id + strict outcome union	< / > / ?；atomically terminalize 最後 key point 並保存 rally-level outcome；不帶新時間 / frame

kind	主要 payload	說明
MOVE_KEY_POINT	key_point_id, playback_cursor	drag/frame step 後由 server 重新解析 authoritative anchor
DELETE_KEY_POINT	key_point_id	draft-only
REOPEN_RALLY	none	清除 terminal 與 outcome、回 OPEN；之後以新的 CLOSE_RALLY 重新關閉
VOID_RALLY	reason	明確作廢 draft，不等於刪除歷史
SUBMIT_RALLY	none	Enter，建立 immutable submission

10.3 Ack

```
{
  "schema_version": "2.0.0",
  "type": "command_ack",
  "command_id": "0190...",
  "room_id": "match:...:capture:...",
  "rally_id": "0190...",
  "operation_kind": "CLOSE_RALLY",
  "result_revision": "28",
  "server_sequence": "1042",
  "effects": {
    "terminal_key_point_id": "0190...",
    "annotation_status": "ready",
    "score_resolution": "resolved",
    "scoring_court_side": "left"
  },
  "resolved_anchor": null
}
```

同一 **command_id** 重送必須回相同結果。對 **CLOSE_RALLY**，effects 必須同時回傳 terminal key point、**ready** 狀態與 strict rally outcome，且 **resolved_anchor** 必須為 null；schema 不允許 score time、score frame 或其他新 event anchor。Server 先寫 DB transaction/outbox，再廣播。

10.4 Conflict

```
{
  "type": "command_rejected",
  "command_id": "0190...",
  "code": "CLOSE_RALLY_TARGET_NOT_LAST",
  "expected_revision": "27",
  "actual_revision": "28",
  "snapshot_refetch_required": true
}
```

CLOSE_RALLY_TARGET_NOT_LAST、**REVISION_CONFLICT**、**CURSOR_STALE**、**CURSOR_IN_GAP**、**RALLY_NOT_OPEN**、**SCORE_PENDING** 等均用明確 domain code。Close target 不再是最後 key point 時必須 CAS 失敗並要求 snapshot refetch，不可自動 terminalize 新點。

11. GraphQL Schema 使用規格

11.0 Code-first 產生規則

- Pothos modules 是 source of truth；建議依 domain co-locate type/query/mutation。
- **server/src/graphql/builder.ts** 只設定 plugins/scalars/context，不定義 domain types。
- **server/src/graphql/schema.ts** 只 import type modules 並 **builder.toSchema()**。
- Prisma generator 同時產生 Prisma Client 與 **prisma-pothos-types**。
- **server/src/graphql/export-schema.ts** 使用 **printSchema(lexicographicSortSchema(schema))** 輸出 SDL。
- CI 先 **prisma generate**，再輸出 SDL、跑 GraphQL Inspector，再執行 Web Codegen。
- API 內部錯誤使用穩定 error code；不要把 DB exception 或 MinIO secret 洩漏到 GraphQL message。

GraphQL 唯一原始碼位於 **server/src/graphql/****，由 Pothos code-first 建構。**packages/contracts/graphql/schema.graphql** 是 CI 產生並提交的 contract artifact，不可手動編輯。以下是前後端共同的核心 shape。

11.1 BigInt

scalar BigInt # *JSON wire representation is a decimal string*

scalar DateTime
scalar JSON

所有 PTS、microseconds、frame index、byte size 使用 BigInt，不使用 GraphQL Int。

11.2 Query

```
type Query {  
  me: User!  
  matches(filter: MatchFilter!): [Match!]!  
  match(id: ID!): Match!  
  captureTimeline(captureSessionId: ID!): CaptureTimeline!  
  rallies(matchId: ID!, filter: RallyFilter, page: PageInput): RallyConnection!  
  rally(id: ID!): Rally!  
  analysisView(input: AnalysisViewInput!): AnalysisView!  
  featureAvailability(matchId: ID!): FeatureAvailability!  
  savedAnalysisViews(matchId: ID!): [SavedAnalysisView!]!  
}
```

11.3 Mutation

```
type Mutation {  
  createMatch(input: CreateMatchInput!): Match!  
  startCapture(input: StartCaptureInput!): CaptureSession!  
  stopCapture(captureSessionId: ID!): CaptureSession!  
  
  applyAnnotationCommand(input: AnnotationCommandInput!): AnnotationCommandResult!  
  createCorrectionDraft(submissionId: ID!): Rally!  
  
  assignTrackIdentity(input: AssignTrackIdentityInput!): TrackIdentityAssignment!  
  retryProcessing(input: RetryProcessingInput!): ProcessingState!  
  saveAnalysisView(input: SaveAnalysisViewInput!): SavedAnalysisView!  
}
```

applyAnnotationCommand 是 WS 不可用時的 fallback，必須走相同 domain handler，不能有另一套語意。

11.4 Rally Snapshot 範例

```
{  
  "id": "018f...",  
  "match_id": "...",  
  "set_id": "...",  
  "court_side_assignment": {  
    "left_team_id": "team-a",  
    "right_team_id": "team-b"  
  },  
  "annotation_status": "READY",  
  "processing_status": "NOT_REQUESTED",  
  "revision": 14,  
  "score_resolution_state": "unknown",  
  "scoring_court_side": null,  
  "scoring_team_id": null,  
  "key_points": [  
    {  
      "id": "...",  
      "sequence_index": 0,  
      "marker_kind": "service",  
      "is_terminal": false,  
      "capture_time_us": "10000000",  
      "capture_frame_index": "599"  
    }  
  ],  
  "latest_submission": null  
}
```

11.5 Feature Availability

```
{  
  "dimensions": {
```

```

    "player": true,
    "team": true,
    "action": false,
    "court_zone": true,
    "direct_outcome": false
  },
  "metrics": [
    "rally_count",
    "event_count",
    "rally_win_rate",
    "source_heatmap",
    "target_heatmap",
    "unresolved_rate"
  ],
  "action_taxonomy": null
}

```

前端不可假設 action / confidence 一定存在。

12. PostgreSQL / Prisma 資料模型

12.1 核心分層

- Match / MatchSet / Team / Player / Roster / CourtSideAssignment。
- CaptureSession / CaptureEpoch / DvrProgram / DvrSegment / MediaAsset。
- RallyDraft (**Rally**) / mutable KeyPoint / AnnotationOperation。
- immutable RallySubmission / RallySubmissionKeyPoint / PointAward。
- ClipJob / ClipKeyPointMapping。
- AiIntegration / AiJob / AiCallbackReceipt。
- AnalysisRun / AnalysisTrack / ContactEvent / Actor / Candidate / BallPathSegment / AnalysisArtifact。
- TrackIdentityAssignment / SavedAnalysisView / OutboxEvent。

12.2 必要欄位與約束

- **CaptureEpoch** 保存 **source_time_base**、epoch 起點 raw PTS、epoch 對應 capture time；raw PTS 允許負值；**DvrSegment** 必須指向 epoch。
- Draft KeyPoint 保存 **capture_epoch_id**、**source_pts**、**capture_time_us**、**capture_frame_index**、**original_playback_cursor** 與 timing precision。
- **RallySubmission** 快照必須保存 **score_resolution_state**；unknown 時 scoring side/team 與 score before/after 允許 null。
- **PointAward** 只存在於 resolved submission，且不可由 unknown submission 建立；保存 score revision before/after，並對 (**set_id**, **score_revision_after**) 建唯一約束以序列化比分 ledger。
- ClipJob、AiJob、AnalysisRun 都直接指向 immutable submission，不得只指 mutable Rally。
- Submitted 資料的 immutable 語意由 service layer、transaction 與可選 DB trigger 共同保障；Prisma schema 本身不足以表達所有 check constraints。
- **track_id** 的 primary key scope 為 (**analysis_run_id**, **track_id**)；**AnalysisTrack.mean_confidence** 為 optional，不得缺值時補 0 或 1。
- **AnalysisRun** 將 AI **producer.name**、**producer.build_id** 與 optional **producer.sdk_version** 保存成可查詢欄位；完整原始結果仍保存為 artifact。
- ContactEvent primary relation 使用 immutable submission key point ID；每個分析 run 每個 key point 恰好一筆；optional **resolved_frame_index** 保存 AI 實際解析事件的 clip frame。
- 每筆已解析 Actor 保存必填 **observation_frame_index**，讓 bbox、footprint 與 **court_pos** 的取樣 frame 不需由 consumer 猜測。
- **RallySubmission.service_key_point_id** 與 **terminal_key_point_id** 指向同一份 immutable submission snapshot 中的 key point；DB 以 unique FK 保證單一引用，service layer 另驗證它們屬於該 submission 且 terminal 為序列最後一點。
- 64-bit 時間、PTS、frame 與 byte size 使用 PostgreSQL BIGINT。

12.3 JSONB 使用限制

JSONB 只適合：provider extensions、尚未固定的 action payload、原始 client cursor audit、saved view filters 與非核心 metadata。以下不得只放 JSONB：ID 關係、revision、score、time anchor、artifact 狀態、job 狀態與可索引的分析主欄位。

13. Clip 與時間碼前處理合約

13.1 Submission → Clip Job

普通使用者不能傳 pre/post-roll。Server 依 match clip profile 建：

```
{
  "clip_job_id": "0190...",
  "rally_submission_id": "018f...",
  "annotation_revision": 14,
  "source_range": {
    "logical_start_time_us": "2823290234",
    "logical_end_time_us": "2838123410",
    "requested_start_time_us": "2818290234",
    "requested_end_time_us": "2848123410"
  },
  "profile_id": "canonical-rally-v1"
}
```

13.2 Canonical Clip

- 保持音訊。
- 固定旋轉與 pixel aspect ratio。
- Canonical FPS/profile 由部署設定；若輸入 VFR，mapping manifest 必須從實際 output sample table 建立。
- Clip media timeline 從 0 開始。
- 每個 submission key point 都要得到：
 - clip_pts
 - clip_time_us
 - clip_frame_index
- 不使用 source_time - start 再乘 FPS 的近似方式產 frame index。

13.3 Timing Manifest

```
{
  "schema_version": "1.0.0",
  "clip_id": "...",
  "source": {
    "capture_session_id": "...",
    "requested_start_time_us": "...",
    "actual_start_time_us": "...",
    "actual_end_time_us": "..."
  },
  "video": {
    "width": 1920,
    "height": 1080,
    "fps": {"num": 60000, "den": 1001},
    "time_base": {"num": 1, "den": 60000},
    "total_frames": "1079",
    "duration_us": "18001333",
    "has_audio": true
  },
  "key_points": [
    {
      "key_point_id": "...",
      "source_pts": "...",
      "capture_time_us": "...",
      "capture_frame_index": "...",
      "clip_pts": "300300",
      "clip_time_us": "5005000",
      "clip_frame_index": "300"
    }
  ]
}
```

13.4 Clip Result

- canonical_clip：AI 分析來源。
- preview_clip：可低 bitrate，但必須有明確 mapping；若 frame count/FPS 相同可直接對應。

- **timing_manifest**。
- SHA-256、byte size、codec metadata。

只有 clip 與所有 key point mapping 驗證通過後，才可建立 AI Job。

14. 外部 AI 串接：保留方法定義，不寫死模型細節

AI 團隊已經有球員追蹤、球追蹤、球場追蹤／投影、個體動作，以及可能存在的群體動作 work。本 repository 不實作 AI，只制定串接目標與結果語意。

14.1 必要方法骨架

1. 讀取 canonical rally clip 與人工 key point anchors。
2. 利用既有 work 取得可用的球員 tracks、球 observation、frame 座標、球場投影與 optional action evidence。
3. 對每個輸入 key point 產生恰好一筆 contact event，不跳過、不重編 ID。
4. 在該 key point 附近建立球與球員的事件歸屬：單人、多位共同參與、多位候選、無法解析，或事件本身不適用球員（例如 terminal 落地／出界）。
5. 以 actor footprint 的 **court_pos** 作代表點；若 terminal 沒有球員，AI 可提供其既有方法產生的 terminal representative court position，或明確回傳缺失。
6. 由相鄰 contact events 建立 A→B path segment。
7. 回傳結構化 Analysis JSON 與逐幀 FlatBuffers overlay。

14.2 不在本規格寫死的事項

- 模型架構、模組執行順序、時間窗大小。
- 球距離、bbox distance、IoU、pose、action 的融合公式或門檻。
- 補點、插值、平滑與 confidence threshold。
- action label 清單、模型原始輸出欄位、群體 phase label。

14.3 球歸屬與多球員重疊

- **resolved_single**：明確一位 actor。
- **resolved_multiple**：AI 明確判定多人共同參與，例如雙人攔網；不是因為 bbox 重疊就自動成立。
- **ambiguous**：有候選但無法決定；**actors=[]**、**actor_candidates[]** 有值。
- **unresolved**：無法提供有效 actor 或候選。
- **no_player**：事件語意本身不需要球員，例如 terminal 球落地／出界；與模型失敗不同。

高 IoU 或球位於重疊框只是一種 evidence。actors 使用陣列以保留多人事件，但 AI 團隊仍需自行判定何時是共同參與、何時只是 ambiguous。

14.4 Action 與群體 phase 待確認

- **action** 為 optional extension；baseline result 不要求。
- label 由 provider 定義，並帶 optional taxonomy ID/version；中央與前端不能硬編碼 serve/receive/set/spike 等清單。
- confidence 也是 optional；沒有就省略，不得假造。
- 群體動作／比賽 phase 目前沒有已確認產品用途，不列第一版 required output。
- AI 團隊後續需回答：真實 label 清單、粒度、confidence 語意，以及 group phase 可支援哪個教練決策。

15. AI Provider API 與 Python SDK

15.1 Provider Capabilities

GET /v1/capabilities 只聲明支援的 contract 版本與 optional extension，不遠端配置模型參數。

```
{
  "schema_version": "1.0.0",
  "provider_name": "team-ai",
  "provider_build_id": "2026-08-07.1",
  "supported_job_schema_versions": ["1.1.0"],
  "supported_result_schema_versions": ["1.0.0"],
  "supported_overlay_formats": ["flatbuffers_v1"],
  "optional_extensions": {
    "action": true,
    "group_phase": false,
  }
}
```

```

    "confidence": false
  },
  "action_taxonomies": []
}

```

15.2 AI Job Request

AI Job Request 的正式版本為 **1.1.0**；**AnalysisResult**、callback envelope、provider capabilities envelope 與 **JobAccepted** 仍各自維持 **1.0.0**。Job 1.1 新增必填的 **clip.download_url_expires_at**，且明確分離 clip signed URL 與 callback bearer token。Capabilities 必須宣告 **supported_job_schema_versions** 包含 **1.1.0** 才能接收。

POST /v1/jobs

Authorization: Bearer <provider-token>

Idempotency-Key: <ai_job_id>

Content-Type: application/json

```

{
  "schema_version": "1.1.0",
  "ai_job_id": "018f6d86-2d8c-7f10-9c1b-0f0c32aa6001",
  "rally_submission_id": "018f6d86-2d8c-7f10-9c1b-0f0c32aa3101",
  "rally_id": "018f6d86-2d8c-7f10-9c1b-0f0c32aa3001",
  "match_id": "018f6d86-2d8c-7f10-9c1b-0f0c32aa1001",
  "annotation_revision": "14",
  "clip": {
    "clip_asset_id": "018f6d86-2d8c-7f10-9c1b-0f0c32aa5001",
    "download_url": "https://media.example/signed/canonical-clip.mp4?...",
    "download_url_expires_at": "2026-08-07T06:00:00Z",
    "sha256": "9a31d43d2c2aeb1a1a98beec4ff8bbd649f68f16eb7345a51fd61a3b302543aa",
    "byte_length": "83124491",
    "content_type": "video/mp4",
    "video": {
      "width": 1920,
      "height": 1080,
      "fps": {"num": 60000, "den": 1001},
      "time_base": {"num": 1, "den": 60000},
      "total_frames": "1079",
      "duration_us": "18001333",
      "has_audio": true
    }
  },
  "key_points": [
    {
      "key_point_id": "...4000",
      "sequence_index": 0,
      "marker_kind": "service",
      "is_terminal": false,
      "clip_pts": "300300",
      "clip_time_us": "5005000",
      "clip_frame_index": "300"
    },
    {
      "key_point_id": "...4010",
      "sequence_index": 1,
      "marker_kind": "contact",
      "is_terminal": true,
      "clip_pts": "480480",
      "clip_time_us": "8008000",
      "clip_frame_index": "480"
    }
  ],
  "outcome": {
    "score_resolution": "resolved",
    "scoring_court_side": "left"
  },
  "callback": {
    "url": "https://central.example/api/v1/ai/callback/018f...",
    "token": "job-scoped-opaque-token",
  }
}

```

```

    "expires_at": "2026-08-07T06:00:00Z"
  }
}

```

Unknown outcome :

```

{"score_resolution":"unknown","scoring_court_side":null}

```

Job 不包含 court definition、team IDs、model requirements、threshold、MinIO upload URI 或中央 DB 設定。球場座標規格是固定 contract，不需要每個 job 重複傳。

clip.download_url 必須是獨立的短期 signed URL，SDK 下載時不得夾帶 **callback.token**。**callback.token** 只用於對中央 callback URL 送出 Bearer token；不得送往 MinIO/S3、不得出現在 browser 或 log。

AI 欄位 ownership

欄位	來源	AI 端行為
ai_job_id	CENTRAL + PASSTHROUGH	result/callback 原樣回傳
rally_submission_id	CENTRAL + PASSTHROUGH	原樣回傳；AI 結果的版本錨點
rally_id, match_id	CENTRAL + PASSTHROUGH	原樣回傳
annotation_revision	SUBMISSION SNAPSHOT + PASSTHROUGH	原樣回傳
clip_asset_id	PREPROCESS + PASSTHROUGH	原樣回傳
clip URL/checksum/video metadata	PREPROCESS	下載並驗證；不回寫修改
key_point_id	SUBMISSION SNAPSHOT + PASSTHROUGH	每個恰好出現一次
marker/sequence/terminal/clip timing	PREPROCESS + PASSTHROUGH	作 anchor，不改寫
outcome	SUBMISSION SNAPSHOT	可使用，不需要在 result 重複回傳
callback URL/token	CENTRAL	token 只授權中央 callback POST；clip 使用獨立 signed URL；兩者皆不放 result、不下發 browser
track/event/action/position	AI_GENERATED	由 AI 輸出

15.3 Provider Accepted

```

{
  "schema_version": "1.0.0",
  "ai_job_id": "...",
  "provider_job_id": "provider-123",
  "state": "accepted",
  "accepted_at": "2026-08-07T05:30:00Z"
}

```

相同 Idempotency-Key 重送應回同一 provider job。

15.4 Python SDK

GitHub 安裝：

```

pip install "volleyball-monitoring-ai-sdk @ git+https://github.com/<owner>/volleyball-monitoring-ai.git@v0.1.0#subdirectory=sdk"

```

SDK 必須提供 Pydantic job/result/capabilities/accepted models、JSON Schema validation、signed clip 下載與 checksum、callback client、FlatBuffers helper、passthrough/invariant validator、FastAPI provider adapter optional extra 與 fake fixtures。SDK 不依賴 torch/CUDA/OpenCV，也不實作 AI 模型。

16. AI Analysis Result Schema

16.1 結果範例

```

{
  "schema_version": "1.0.0",
  "analysis_id": "analysis-0190",
  "analysis_version": "provider-build-2026-08-07.1",

```

```

"ai_job_id": "018f...6001",
"rally_submission_id": "018f...3101",
"rally_id": "018f...3001",
"match_id": "018f...1001",
"annotation_revision": "14",
"clip_asset_id": "018f...5001",
"input_clip_sha256": "9a31d43d2c2aeb1a1a98beec4ff8bbd649f68f16eb7345a51fd61a3b302543aa",
"producer": {"name": "team-ai", "build_id": "2026-08-07.1", "sdk_version": "0.1.0"},
"tracks": [
  {"track_id": 8, "court_side": "left", "first_frame_index": "0", "last_frame_index": "1078"}
],
"contact_events": [
  {
    "key_point_id": "...4000",
    "sequence_index": 0,
    "marker_kind": "service",
    "is_terminal": false,
    "anchor_frame_index": "300",
    "resolved_frame_index": "301",
    "association_state": "resolved_single",
    "actors": [{
      "track_id": 8,
      "observation_frame_index": "301",
      "frame_bbox": {"x1": 0.12, "y1": 0.30, "x2": 0.21, "y2": 0.81},
      "frame_foot_pos": {"x": 0.165, "y": 0.81},
      "court_pos": {"x": -0.06, "y": 0.24}
    }],
    "actor_candidates": [],
    "ball": {"state": "observed", "sample_frame_index": "301", "frame_pos":
      ↪ {"x": 0.182, "y": 0.614}},
    ↪ "representative_court_positions": [
      {"track_id": 8, "basis": "player_footprint_proxy", "court_pos": {"x": -0.06, "y": 0.24}}
    ],
    "quality_flags": []
  }
],
"path_segments": [],
"summary": {"track_count": 1, "contact_event_count": 1, "path_segment_count": 0, "unresolved_
  ↪ event_count": 0},
"extensions": {}
}

```

16.2 Passthrough 與 1:1 invariant

- ai_job_id/rally_submission_id/rally_id/match_id/annotation_revision/clip_asset_id 原樣回傳。
- 每個 input key point 恰好一個 contact event。
- key_point_id/sequence_index/marker_kind/is_terminal 與 job 完全一致。
- N 個 events 通常有 N-1 個 segments；只有單一 service 且 terminal 的回合可為 0 段。
- 無法判斷 actor 仍回 event，不得刪掉。

16.3 Actor observation frame

每個已解析 actor 必須帶 **observation_frame_index**，表示 **frame_bbox**、**frame_foot_pos** 與 **court_pos** 取樣的 clip frame。AI 可在人工 anchor 附近選擇更可靠的 frame，但不得讓 consumer 猜測這些座標來自哪一幀。

AI wire result 不需要生成跨系統 **segment_id**；每段路徑以 **sequence_index + start_key_point_id + end_key_point_id** 識別，中央 ingest 後才建立自己的 DB ID。**producer.sdk_version** 為 optional，因 AI provider 可不用 SDK 實作同一 contract。

16.4 Association state

state	actors	candidates	語意
resolved_single	1	0	明確單人
resolved_multiple	>=2	0	明確共同參與

state	actors	candidates	語意
ambiguous	0	>=1	有候選但無法判定
unresolved	0	0	模型無法解析
no_player	0	0	terminal 落地／出界等本來就無球員

`actors[]`、`actor_candidates[]` 的 association confidence 均 optional。Action 可放在 actor 中作 optional provider-defined extension；沒有 action 時省略。

16.5 座標

- `frame_pos`：影像平面正規化點，x/y 在 0..1。
- `frame_bbox`：影像平面正規化框， $x1 \leq x2$ 、 $y1 \leq y2$ 。
- `frame_foot_pos`：影像中的球員腳底代表點。
- `court_pos`：AI 已投影的球場座標；球場內 x/y 0..1，但場外允許負值或 >1，不可 clamp。
- 固定球場定義：x=0 左端線、x=1 右端線（18m）；y=0 畫面標準化球場上側邊線、y=1 下側邊線（9m）。顯示層可翻轉，資料層不可因教練視角改寫。
- 缺值用 null / 省略與 quality flag，不用 (0,0) 代表 unknown。

16.6 Path Segment

Segment 只引用相鄰 key point 並保存起終代表點陣列；多人事件可有多個 position。render_state=complete/partial/unavailable 說明能否畫線。Rally outcome 由中央 submission join，不在每段重複；避免 AI result 與人工得分互相矛盾。

17. AI Callback

17.1 Token

- Callback URL job-specific。
- Bearer token 只對該 job 有效，DB 只保存 hash。
- 可在 TTL 內重試，不是用一次即銷毀。
- `callback_id` 每次 callback 唯一；相同 callback ID 重送回相同結果。

17.2 Progress / Failed

```
{
  "callback_id": "0190...",
  "kind": "progress",
  "schema_version": "1.0.0",
  "ai_job_id": "...",
  "progress": 0.55,
  "phase": "provider-defined optional string",
  "message": null
}

{
  "callback_id": "0190...",
  "kind": "failed",
  "schema_version": "1.0.0",
  "ai_job_id": "...",
  "error": {
    "code": "PROVIDER_PROCESSING_FAILED",
    "message": "...",
    "retryable": true
  }
}
```

中央前端不能依 phase 作狀態機，只依 kind/progress。

17.3 Completed Multipart

```
POST /v1/ai/callback/{ai_job_id}
Authorization: Bearer <callback-token>
Content-Type: multipart/form-data
```

Parts：

- **metadata** : callback envelope JSON , 含 callback ID 與 checksums 。
- **analysis** : **application/json** 。
- **overlay** : **application/vnd.volleyball.overlay+flatbuffers;version=1** 。

中央以 streaming 方式寫 temporary object , 不將整個 overlay 載入 Node heap 。全部驗證通過後才 commit analysis run 。

17.4 HTTP Semantics

Status	語意
200	已處理或相同 callback id 已處理
202	接受 , artifact ingestion 仍在背景進行
401/403	token 錯誤 / 過期
409	callback 與 job/submission 版本衝突
413	payload 過大
415	content type/FlatBuffers version 不支援
422	Schema/invariant 失敗 , 不可重試原 payload
503	暫時不可用 , 可 retry

18. FlatBuffers Overlay v1

18.1 原則

- AI 回傳一個 rally 完整 sequence binary 。
- Central 驗證後依固定 frame chunk 切成 Web 用 chunks 。
- 採 Structure of Arrays , 避免大量 nested JS object 。
- frame 座標量化 U16 ; court 座標 float32 , 因為可場外 。
- 缺值由 flags/sentinel 表示 , 不偽造 0 。

18.2 Chunk 內容

```
start_frame_index
frame_count
frame_offsets[frame_count + 1]
track_ids[detection_count]
bbox_x1/y1/x2/y2[detection_count]
foot_x/foot_y[detection_count]
court_x/court_y[detection_count]
action_label_ids[detection_count]
confidence_q[detection_count]
flags[detection_count]
```

```
ball_x/ball_y[frame_count]
ball_confidence_q[frame_count]
ball_flags[frame_count]
```

Invariant :

- **frame_offsets[0] = 0**
- last offset = detection count 。
- 所有 detection column 長度相同 。
- action label id **65535** 表示缺少 。
- confidence **-1** 表示缺少 。
- court validity 由 flag 表示 ; x/y 本身允許負值或 >1 。

18.3 Web Manifest

```
{
  "schema_version": "1.0.0",
  "analysis_run_id": "...",
  "overlay_version": 2,
  "video": { "width": 1920, "height": 1080, "total_frames": "1079" },
  "chunk_frame_count": 120,
  "action_dictionary": [],
```

```

"chunks":[
  {
    "index":0,
    "start_frame":"0",
    "end_frame":"119",
    "url":"/v1/analysis-runs/.../overlay-chunks/0",
    "byte_length":"48211",
    "sha256":"..."
  }
]
}

```

Web 只預載當前與下一 chunk；seek 時取消無用 request 並載目標 chunk。

19. 教練／裁判端 UX

19.1 Routes

/	場次選單／最近紀錄
/matches/:matchId/live	現場總覽
/matches/:matchId/paths	球路與熱點
/matches/:matchId/players	球員列表與個人頁
/matches/:matchId/stats	統計與篩選
/matches/:matchId/history	Rally / 保存視圖歷史
/matches/:matchId/replay/:rallyId	單 Rally 回放
/annotate/:matchId	標註工作台
/settings	系統設定（依角色）

19.2 現場頁

- 場次、比分、局數、live edge 與最新完成分析。
- 最新 rally 建議卡；必須附資料來源與樣本數。
- 快速進入最新回放。
- 底部導覽固定但不遮住播放器/球場。

19.3 球路頁

篩選：

- Team。
- Player（identity mapping 存在時）。
- Set / 時間範圍。
- Start/target court zone。
- Rally outcome。
- Association quality。
- Action（feature available 才顯示）。

2D court 顯示 A/B、方向、熱點；點擊路徑開對應 rally/time。

19.4 回放頁

- Video + transparent Canvas overlay。
- 2D court 同步。
- 全部符合篩選路徑淡灰，當前 segment 高亮。
- Overlay modes：
 - Off
 - Tracking
 - Coach
 - Tactical
 - Debug（只對授權角色）
- Layer toggle：bbox、track ID、player、action、ball、trail、footprint、confidence。
- Track ID 在未綁 player 時顯示 **Track 8**，不能冒充背號。

19.5 歷史與保存紀錄

- 頂部返回場次選單。

- History 可按 set、score、日期、processing state、quality 篩選。
- Saved Analysis View 保存 filter/query 設定，不保存一份過時統計快照；開啟時用當前資料重算，顯示 saved at/schema version。

19.6 Offline / Reconnect

- WS 中斷仍可看已載入影片/資料，但新增標記要顯示 pending。
 - Command 先放 local outbox，帶 command ID。
 - 重連後比較 server revision。
 - Mapping window 過期時，尚未送出的 marker cursor 必須重新 resolve 或要求使用者確認。
-

20. 分析與統計

20.1 Baseline 一定可做

只依人工 outcome、AI contact association 與 court positions：

- Rally count。
- Team rally win/loss（排除 unknown）。
- Contact event count。
- Participant event count。
- Source/target heatmap。
- A→B 方向分布。
- 區域轉移 matrix。
- Terminal 位置分布。
- Ambiguous/unresolved 比例。
- Samples 數、資料品質與缺值率。

20.2 Identity 後可做

- 每球員 contact 數。
- 每球員起點／目標熱點。
- 每球員 rally participation 與 rally outcome split。
- 跨 rally 球員路徑。

20.3 Action taxonomy 後才可做

- Attack / serve / receive 等 action filter。
- Kill/error/ace/direct outcome。
- Hitting efficiency。
- Setter distribution。
- Side-out / breakpoint 細分。

在 direct outcome contract 未確認前，不能把「包含此事件的 rally 最後贏球」當成該事件直接得分。

20.4 API 回應

每個 metric 必須附：

- value
- sample_count
- excluded_count
- unknown_count
- quality_breakdown
- feature_dependencies

教練卡片不能只顯示一個無樣本數百分比。

21. 後端／前端實作優先順序

Phase 0 — Contract Freeze 與 Scaffold

Contracts / SDK Agent

- 建 GraphQL、REST、WS、AI JSON Schema、FlatBuffers。
- 建正常、缺球、多 actor、ambiguous、terminal、場外 service fixtures。
- SDK models 與 validator。

Backend Agent

- Prisma schema、Compose、health、MinIO init、MediaMTX baseline。
- 不先實作完整功能。

Luna Frontend Agent

- Nuxt shell、routes、bottom/top nav、design tokens。
- Fixture-driven player/timeline/court prototype。

Exit

- 相同 fixtures 通過 TS/Python schema validation。
- Directory ownership 不衝突。
- ADR 紀錄 API boundary。

Phase 1 — Core Domain / Auth / DB

Backend：

- Match/team/player/set/side assignment。
- Prisma migration。
- GraphQL Yoga endpoint。
- 開發 auth 與 role guard。

Frontend：

- 場次選單、history shell、route guards。

Exit：可建立場次、set、左右隊與 roster；換邊資料可查。

Phase 2 — Media Ingest / 完整 Timeline

Backend / Media：

- MediaMTX ingest/record。
- Segment index → MinIO/DB。
- Timeline GraphQL。
- Live rolling playback window。
- Archive window / lazy seek。
- PlaybackCursor resolve/frame step。

Frontend：

- Live video/audio。
- Full timeline、live edge、gap。
- Seek 遠端 window 與回到 live。
- requestVideoFrameCallback cursor。

Exit：一小時以上 capture 可持續錄；client 記憶體不隨整場線性成長；任意已保存 frame 可 resolve。

Phase 3 — Annotation Vertical Slice

Backend：

- WS command/revision/idempotency。
- Draft mutable model + operation log。
- Z/Space/CLOSE_RALLY(left/right/unknown)/reopen/move/delete/Enter。

- immutable submission。

Frontend：

- 剪輯式 timeline 與 tracks。
- 灰／綠 mask、score strip、快捷鍵模式。
- 多人 presence/conflict/reconnect。

Exit：兩個 iPad 同時標；refresh/reconnect 後一致；close 不建 timestamp／score frame；unknown 可提交。

Phase 4 — Clip／Fake AI／SDK

- Clip worker 與 timing manifest。
- AI Integration/capabilities/job/callback。
- SDK v0.1.0 可 GitHub 安裝。
- Fake provider 回 fixtures。

Exit：Enter 後自動 clip → fake AI → callback → analysis run。

Phase 5 — Coach Replay／Overlay

- Overlay sequence ingest/chunk。
- Manifest REST。
- Canvas overlay。
- 2D court、A/B path 與 history。

Exit：點路徑可跳影片；seek 只載附近 chunks；action 缺失不破版。

Phase 6 — Identity／Analytics

- Track-player mapping。
- Baseline metrics/filters/saved views。
- Quality 與 sample count。

Exit：跨 rally player view 不依賴 track ID；side switch 後 team 統計正確。

Phase 7 — Hardening

- 2-3 小時 ingest soak。
- Source reconnect/discontinuity。
- Postgres/Redis/MinIO/server/worker restart。
- Callback duplicate/retry/bad checksum。
- iPad memory/performance。
- Backup、retention、metrics、audit。

22. 三個 Subagent 的協作規則

22.1 固定 Ownership

Agent	可主動修改
A Contracts/SDK	<code>packages/contracts/**, sdk/**</code>
B Backend/Media/Infra	<code>server/**, worker/**, packages/db/**, infra/**</code>
C Luna Frontend	<code>web/**</code>
Main Agent	root、docs、CI、跨目錄整合與 contract 批准

任何 Agent 若需改別人責任範圍，先回報主 Agent，不直接動手。

22.2 第一輪任務

Agent A

建立 v1 contract baseline 與 Python SDK skeleton。只實作 Schema、validation、fake provider adapter 與 fixtures，不實作 AI 模型。完成後回報 contract 版本、生成物、測試與未定 action/phase 事項。

Agent B

建立 TypeScript server/worker、Prisma、PostgreSQL、MinIO、Redis、MediaMTX 與 Compose skeleton。GraphQL 使用 Yoga，binary 走 REST，高頻 annotation 走自訂 WS。不得自行發明與 contracts 不同的欄位。

Agent C - Luna Frontend

建立 Nuxt 4 多頁 shell、底部導覽、頂部狀態列、annotation/coach/history route 與 fixture-driven 元件。只參考舊 MVP UI libraries/framework，不沿用 domain flow/schema/statistics。

22.3 每次交付格式

Completed

- ...

Changed files

- ...

Contract/schema version consumed

- ...

Tests run

- command + result

Known issues / blockers

- ...

Decisions needed from main agent

- ...

22.4 Main Agent Merge Gate

- Contract 變更先於實作。
- Fixture 要同時由 TS/Python 讀取。
- Prisma migration 與 GraphQL field 一致。
- UI field 必須由 generated type 取得。
- 每一 Phase 都有可執行的 end-to-end evidence。
- 不接受「已建立 component/table」作為 flow 完成證據。

23. 主 Agent 可直接使用的 Prompt

你現在位於新 Repository `volleyball-monitoring-ai` 的 root。

你的身份是主要 PM Agent、系統架構師與最終整合者。先完整閱讀：

1. `docs/MASTER\_IMPLEMENTATION\_SPEC.md`
2. `AGENTS.md`
3. `packages/contracts/README.md`
4. 現有 git status、目錄與 placeholder

本專案不實作 AI 模型，只實作：

- Nuxt 標註端與教練端
- Fastify + GraphQL Yoga + Pothos/Prisma 的 GraphQL/REST/WebSocket 中央後端
- MediaMTX + FFmpeg 完整 DVR、windowed lazy playback 與 clip 服務
- Prisma/PostgreSQL、MinIO、Redis 與 durable workers
- 外部 AI Job/callback contracts
- 可從 GitHub subdirectory 安裝的 Python SDK
- Fake AI Provider 與端到端 fixtures

最多同時派出三個 subagent，固定分工：

- A. Contracts + Python SDK
- B. Backend + Media + Infra
- C. Nuxt Frontend

你必須保留架構與 Schema 決策權。不要讓 subagent 修改彼此責任目錄；跨目錄變更由你完成。

執行規則：

- 先稽核 `scaffold` 是否與 `MASTER spec` 一致，再依 `Phase 0` 開始。
- `Contract-first`；跨系統欄位只以 ``packages/contracts`` 為真實來源。
- 不要寫死 `action labels`、`confidence`、`AI` 模型參數或群體 `phase`。
- ``CLOSE_RALLY`` 只將指定的 `server-confirmed` 最後 `key point` 標 `terminal` 並保存 `rally-level outcome`，
→ 不建立時間點或得分 `event`。
- ``?`` 是明確 `unknown`，可送出；`pending` 不可送出。
- `Left/right` 只表示 `court side`，不是 `team identity`。
- `Browser` 只送 `PlaybackCursor`；`authoritative PTS/frame` 由後端 `resolve`。
- 整場可回放，但 `client` 只 `lazy-load` 目前數分鐘，不下載全場。
- `GraphQL` 管 `structured domain data`；`REST` 管 `media/binary/callback`；`WebSocket` 管 `annotation`
→ `command/revision`；`FlatBuffers` 管逐幀 `overlay`。
- `Submitted revision` `immutable`。
- `Tracker ID` 只在單 `rally` 有效。
- `AI` 系統只透過 `Job/Result/Callback/SDK` 串接。

先建立/確認 `baseline commit`。接著提出 `Phase 0` 三個 `subagent` 的精確任務、路徑 `ownership` 與 `exit`
→ `criteria`，派出最多三個 `subagent`。你自己同時負責 `root/docs/CI` 與整合測試。

每個 `Phase` 結束：

- 實際執行測試與 `build`
- 更新 ``docs/progress.md``
- 列出完成 `flow`、未完成 `flow`、風險與下一 `Phase`
- 建立小而清楚的 `commit`，不製造一個巨大 `commit`

不得把 `placeholder`、`mock` 畫面或單一 `layer` 宣稱成已完成。完整 `flow` 至少需具備：`DB/migration` → `domain/`
→ `service` → `GraphQL/REST/WS` → `frontend` → `success/error/reconnect` → `test`。

24. Luna Worker 定義與建立 Prompt

24.1 建議 TOML

```
name = "luna_worker"
description = "Execution-focused worker for one bounded repository task with explicit path
→ ownership, acceptance criteria, and no authority to alter architecture or public
→ contracts."
model = "gpt-5.6-luna"
model_reasoning_effort = "medium"
sandbox_mode = "workspace-write"

developer_instructions = ""
Work only on the exact task delegated by the parent agent and only in the explicitly assigned
→ files or directories.
```

Before editing:

1. Read the nearest `AGENTS.md` files.
2. Inspect the actual current implementation; do not assume the parent prompt is still
→ accurate.
3. Restate the task boundary, owned paths, consumed contract/schema version, and acceptance
→ criteria internally before making changes.

You may perform bounded implementation, file discovery, data organization, fixture work,
→ focused tests, documentation updates, and small fixes that are fully specified by the
→ parent.

You must not:

- Change product goals, architecture, public contracts, database semantics, or cross-team
→ APIs.
- Add, remove, or upgrade dependencies unless explicitly authorized.
- Modify files owned by another active worker.
- Refactor unrelated code, broaden scope, or create speculative abstractions.
- Spawn additional subagents.
- Implement AI models or invent action labels, confidence semantics, or model parameters.
- Hide blockers by guessing.

If the task requires a cross-cutting decision, contradicts the current contract, lacks
→ required input, or would touch unowned paths, stop and report the exact blocker to the
→ parent agent.

Use the smallest complete change. Run focused validation for changed behavior. Do not claim a
→ test was run unless you actually ran it.

Return exactly these sections:

- Completed
- Changed files
- Contract/schema version consumed
- Tests run (commands and results)
- Known issues/blockers
- Decision needed from parent (or None)

```
"""
```

`sandbox_mode=workspace-write` 是合理的，因 worker 需要實際完成小型修改；若某個 worker 只做搜尋/審查，主 Agent 可在 spawn 時改用 `read-only agent`。

24.2 建立 Prompt

請建立個人層級 Codex 自訂 Agent：

```
~/codex/agents/luna-worker.toml
```

先使用目前已安裝的 Codex CLI 執行 ``codex --version`` 與 ``codex --help``，並查閱該版本可用的本機/官方
→ `custom agent` 設定格式；不要假設舊格式仍相同。

請建立以下語意的 Agent：

- `name = "luna_worker"`
- `model = "gpt-5.6-luna"`
- `model_reasoning_effort = "medium"`
- `sandbox_mode = "workspace-write"`
- `description` 與 `developer_instructions` 採本 repository ``docs/MASTER_IMPLEMENTATION_SPEC.md``
→ 中的 Luna Worker 定義；若目前版本欄位名稱不同，只做格式相容調整，不改變行為邊界。

`luna_worker` 只負責由主 Agent 明確委派、邊界清楚、具備 `path ownership` 與驗收條件的執行型任務。它不得修改產
→ 品目標、架構、`public schema`、`database` 語意或未授權 `dependency`；不得擴張範圍或再派 `subagent`。遇到跨
→ 團隊決策或缺少 `contract` 時必須停止並回報。

請保留現有其他 Codex 設定，不覆蓋或刪除無關內容。若目錄或檔案不存在才建立。

建立完成後：

1. 顯示新增檔案完整內容。
2. 顯示只針對該檔案的 `diff`。
3. 使用目前系統可用的 TOML parser 驗證語法。
4. 確認 ``name`` 是 Codex 辨識自訂 Agent 的來源，並確認檔案位於個人 Agent 目錄。
5. 在新的 Codex session 中以最小的 `read-only` 測試任務要求主 Agent `spawn `luna_worker``；不要用不存在於
→ 目前版本 `help` 中的虛構 CLI 子命令。
6. 回報版本、驗證命令、驗證結果與任何相容性調整。
7. 不修改任何與此 Agent 無關的設定。

24.3 Project Config

`#:schema https://developers.openai.com/codex/config-schema.json`

```
[agents]
```

```
enabled = true
```

```
max_concurrent_threads_per_session = 3
```

25. 測試與 Definition of Done

25.1 Contract

- JSON Schema fixtures 通過 Python 與 TypeScript validator。
- FlatBuffers file identifier/version 不符會 fail。

- Passthrough ID 改動會 fail。
- Action 完全缺失仍通過。
- Confidence 缺失仍通過。

25.2 Media / Time

- 30fps、59.94fps、VFR input。
- 一小時以上 live ingest。
- Source reconnect/discontinuity。
- Timeline gap 顯示。
- 遠端 seek 建立新 playback window。
- Client memory 不隨整場時間線性成長。
- PlaybackCursor resolve 在目標呈現 frame 可重現。
- Frame step 前後一幀正確。

25.3 Annotation

- Z/Space/left-close/right-close/unknown-close/Enter 六個固定語意。
- **CLOSE_RALLY** 不新增 timestamp、score frame 或 score event，terminal anchor 沿用 target key point。
- 只有 service 的 service error。
- Unknown 可提交，pending 不可。
- Reopen 與 change terminal。
- Duplicate command idempotency。
- 多人 revision conflict/refetch。
- Submitted immutable/correction draft。

25.4 Clip / AI

- pre/post-roll 被 source 開始/結束截斷。
- 所有 submission key point 有 clip mapping。
- AI Job 從 SDK model 解析。
- Clip checksum 錯誤。
- Capabilities 不相容。
- Job retry/idempotency。
- Callback duplicate、過期 token、bad schema、bad FlatBuffers、oversize。
- 1:1 key point invariant。
- resolved multiple 與 ambiguous 分開。

25.5 Coach

- Bottom nav 與歷史。
- Video overlay 可切層。
- 2D path 與 video seek 同步。
- Track 未綁 player 時不顯示假背號。
- Action unavailable 時 filter/metric 隱藏。
- Unknown outcome 排除 win rate。
- 換邊後 team 統計正確。
- 樣本數與品質顯示。

25.6 第一版完成條件

第一版只有在以下全部成立時才完成：

- 伺服器可持續錄整場且前端可任意回看已保存區間。
- Client 只 windowed lazy-load，live 持續錄製。
- Key point authoritative time 對齊影片呈現 frame。
- 多人標註與 revision 恢復正確。
- 灰/綠 mask 與 score state 符合使用流程。
- Enter 建立 immutable submission。
- Clip mapping 正確。
- SDK 可由 GitHub 安裝，Fake 與真 AI 使用同一 contract。
- Callback/FlatBuffers 可驗證、保存與重試。
- 教練端多頁、歷史、overlay、A/B 與 baseline 分析可用。
- 所有 durable state 位於 PostgreSQL/MinIO；API/worker 重啟不遺失。

26. 舊 MVP 可參考與禁止範圍

只允許參考：

- Nuxt 4 / Vue 3 / TypeScript。
- Vant/Tailwind/Motion/Lucide 等 UI library。
- GraphQL Yoga + Pothos + Prisma + PostgreSQL + MinIO + FFmpeg 的工程組合。
- Binary upload / stream 不走 GraphQL 的分工。
- WebSocket command ID / server revision 概念。
- Docker Compose、Kubernetes、health endpoint、虛擬化列表等工程作法。

禁止沿用：

- 舊手動球員/事件/球路輸入流程。
 - 舊 action enum、統計公式或比分/輪轉語意。
 - 舊 DB schema 與 memory store。
 - 舊頁面資訊架構或 demo data。
 - 任何與本文件 fixed semantics 衝突的實作。
-

27. 尚待人類／AI 團隊確認，但不阻塞 Baseline

1. GitHub owner 與正式 repository URL。
2. 正式 auth/SSO。
3. 第一優先直播輸入協議與 capture 硬體。
4. Raw segment、playback window、clip、analysis retention。
5. Canonical clip FPS/解析度。
6. AI action taxonomy、粒度、confidence。
7. 群體 phase 是否有產品用途。
8. AI result 人工 review UI 是否第一版需要。
9. 正式教練 recommendation 規則。
10. 賽事比分是否需完整 set 勝負／暫停／換人功能；baseline 至少保存 rally outcome 與 score snapshot。

主 Agent 不得為了看起來完整而擅自填補這些未知項目。

28. 官方工程參考與選型依據

- Codex custom agents / subagents: <https://developers.openai.com/codex/agent-configuration/subagents>
 - Codex config reference: <https://developers.openai.com/codex/config-reference>
 - Nuxt 4: <https://nuxt.com/docs/4.x/>
 - GraphQL Yoga: <https://the-guild.dev/graphql/yoga-server>
 - Prisma PostgreSQL: <https://www.prisma.io/docs/orm/overview/databases/postgresql>
 - MediaMTX: <https://mediamtx.org/docs/>
 - MediaMTX playback: <https://mediamtx.org/docs/features/playback>
 - HTMLVideoElement requestVideoFrameCallback: <https://developer.mozilla.org/docs/Web/API/HTMLVideoElement/requestVideoFrameCallback>
 - FlatBuffers: <https://flatbuffers.dev/>
 - pip VCS/subdirectory: <https://pip.pypa.io/en/stable/topics/vcs-support/>
-

v3.2 交付註記

本 ZIP 是可供主 Agent 啟動 Phase 0 的「contract-first scaffold」，不是已完成產品。它包含可驗證的 Schema、fixtures、Python SDK 模型、Prisma 資料模型、Nuxt PWA shell、Fastify/Yoga/Pothos shell、worker 角色與 Docker/Traefik 骨架；media ingest、annotation persistence、clip pipeline 與 coach overlay 仍須依第 21 章逐階段完成。不得把 placeholder、echo WebSocket 或 route shell 宣稱為 vertical slice 完成。

本規格已依實作與使用流程重新稽核；starter repository 只建立 contract-first 骨架與可驗證 fixtures，不代表產品功能已完成。主 Agent 必須依 Phase 逐條完成 vertical slice，不得把 placeholder 當成果。

29. 最終欄位稽核與 Team Contract Catalog

本章是開工前的最後欄位檢查。若前文範例與本章衝突，以本章、repository 中的 versioned schema 與 golden fixtures 為準。

29.1 三個團隊的 Schema 交付

Producer	Contract	Consumer	Transport	Source of truth
前端／後端	PlaybackWindow request/descriptor	後端／前端	REST JSON	<code>packages/ contracts/ media/ playback-window-request.schema.json</code> + <code>playback-window-descriptor.schema.json</code>
前端	PlaybackCursor	後端	WS/REST JSON	<code>packages/ contracts/ media/ playback-cursor.schema.json</code>
前端／後端	Canonical FrameStep	後端／前端	REST JSON	<code>packages/ contracts/ media/ frame-step-request.schema.json</code> + <code>canonical-frame-anchor.schema.json</code>
後端	ResolvedMediaAnchor	前端/DB	WS/REST JSON	<code>packages/ contracts/ media/ resolved-media-anchor.schema.json</code>
前端	AnnotationCommand	後端	WebSocket JSON	<code>packages/ contracts/ annotation/ realtime.schema.json</code>
後端 AI	Ack/Conflict/Snapshot ProviderCapabilities	前端 後端	WebSocket + GraphQL REST JSON	schema + Pothos SDL <code>packages/ contracts/ ai/ capabilities.schema.json</code>
後端前處理	AIJobRequest	AI SDK/provider	REST JSON	<code>packages/ contracts/ ai/ job.schema.json</code>
AI	JobAccepted	後端	REST JSON	<code>packages/ contracts/ ai/ job-accepted.schema.json</code>
AI	AnalysisResult	後端	callback JSON part	<code>packages/ contracts/ ai/ result.schema.json</code>
AI 後端	OverlaySequence OverlayManifest/ Chunk	後端 前端	callback FlatBuffers part REST JSON + FlatBuffers	<code>flatbuffers/ overlay.fbs</code> manifest schema + <code>overlay-chunk.fbs</code>
後端	Domain query/ mutation	前端	GraphQL	Pothos code-first generated SDL

29.2 前端送後端的 PlaybackCursor

欄位	型別	必填	Ownership	說明
<code>playback_window_id</code>	UUID/string	是	CLIENT_OBSERVED + PASSTHROUGH	當前播放器 window
<code>mapping_version</code>	integer	是	BACKEND 產生、client passthrough	防止過期 mapping
<code>player_media_time_us</code>	decimal string	是	CLIENT_OBSERVED	該 window 的 player timeline
<code>observation_enum</code>		是	CLIENT_OBSERVED	rvfc 或 fallback
<code>presented_frames</code>	decimal string/null	否	CLIENT_OBSERVED	只供 audit/掉幀診斷
<code>seek_generation</code>	integer	是	CLIENT_OBSERVED	避免舊 frame callback
<code>cursor_status</code>	enum	是	CLIENT_OBSERVED	ready 才允許標記

不應從 browser 傳 `source_pts`、`capture_time_us` 或 `capture_frame_index` 作 canonical 值。

29.3 後端解析後的 KeyPoint Anchor

欄位	型別	Ownership	說明
capture_epoch_id	UUID	BACKEND_DERIVED	raw PTS 有效範圍
source_pts	decimal string	BACKEND_DERIVED	epoch 內來源 PTS
source_time_base	rational	BACKEND_DERIVED	PTS 單位
capture_time_us	decimal string	BACKEND_DERIVED	整場 canonical 時間
capture_frame_index	decimal string	BACKEND_DERIVED	整場 canonical frame
timing_precision	enum	BACKEND_DERIVED	frame_exact/pts_exact/estimated
snap_distance_us	decimal string/null	BACKEND_DERIVED	client observation 到 sample 距離
original_playback_cursor	JSON audit	CLIENT_OBSERVED	不作 canonical 查詢欄位

29.4 Immutable Submission Snapshot

必要：submission ID、rally ID、annotation revision、content hash、score resolution、nullable scoring court side/team、left/right team snapshot、nullable scores、service/terminal key point IDs、clip policy version 與實際 pre/post-roll 快照、submitted by/at、supersedes ID 與完整 submission key points。

unknown submission：score resolution=unknown；scoring side/team、score before/after 與 PointAward 皆為 null/不存在。**resolved** submission：side/team 與 score ledger 必須完整且 transaction 一致。

29.5 AI Job / Result 最小原則

- Job 只傳已裁切影片、局部 key points、人工得分解析與 callback。
- Job 不傳 court definition、requirements、模型參數、MinIO prefix 或 team metadata。
- Result 不重複人工得分、side assignment 或 team truth；中央以 submission join。
- 所有跨 AI 邊界 ID 在欄位表標成 PASSTHROUGH。
- AI 生成欄位只包含 tracks、contact events、path segments、overlay 與 optional extensions。

29.6 已知待確認但不阻塞介面

1. AI action 真實 taxonomy、粒度與 confidence。
2. 群體動作／比賽 phase 是否有可驗收產品用途。
3. 第一個正式來源 protocol 與 canonical recording profile。
4. pre/post-roll 預設秒數。
5. production auth provider 與 retention policy。

這些不得被 subagent 猜成既定事實。先以 optional extension、設定值或 ADR open decision 保留。

30. v3.2 最終交付與啟動

Repository ZIP 內必須同時包含：

- docs/SYSTEM_SPEC_V3_2.md
- docs/SYSTEM_SPEC_V3_2.tex
- docs/SYSTEM_SPEC_V3_2.pdf
- docs/MAIN_AGENT_PROMPT.md
- CODEX_SOL_PROMPT.txt
- .codex/agents/contracts-sdk-worker.toml
- .codex/agents/backend-worker.toml
- .codex/agents/luna-worker.toml
- 完整 starter 目錄、contract fixtures、Python SDK、Compose 與 Traefik 設定。

本地啟動：

```
cp .env.example .env
docker compose -f infra/compose.yaml --profile app --profile dev-ai up --build
```

iPad 以 Safari 開啟 Docker host 的 LAN IP 或 DNS，確認 PWA、WebSocket、GraphQL、HLS 均為同源。Traefik :8080 dashboard 與 MinIO :9001 只供本地開發，不是正式公開入口。